

Sažetak

Naslov: Pouzdana obrada IoT telemetrije u mikroservisnoj arhitekturi vođenoj događajima primjenom protokola MQTT

Sažetak:

U radu je opisano oblikovanje i implementacija prototipa sustava za pouzdanu obradu telemetrije Interneta stvari u mikroservisnoj arhitekturi vođenoj događajima. Sustav povezuje mikroupravljače ESP32 s pridruženim DHT11 senzorima, posrednika Eclipse Mosquitto te dva neovisna mikroservisa razvijena u okviru Spring Boot, koji mjerenja temperature i vlažnosti primaju protokolom MQTT i pohranjuju ih u bazu PostgreSQL proširenu ekstenzijom TimescaleDB. Težište rada je slojevita strategija pouzdanosti: razina kvalitete usluge QoS 1 jamči dostavu barem jednom, a moguće duplikate uklanja aplikacijska deduplikacija temeljena na pohrani Redis i jedinstvenom identifikatoru poruke. Dodatnu otpornost na prekide veze osiguravaju perzistentna sesija posrednika i međuspremnik u radnoj memoriji uređaja. Eksperimentalno vrednovanje kroz šest scenarija, uključujući kvar posrednika, kvar servisa i visoko opterećenje, pokazalo je nulti gubitak poruka, ispravno suzbijanje duplikata te medijan latencije ispod 20 ms pri stabilnom opterećenju, čime je potvrđena djelotvornost predloženog pristupa.

Ključne riječi: Internet stvari, MQTT, mikroservisi, deduplikacija, pouzdanost, QoS

Summary

Title: Reliable IoT telemetry processing in an event-driven microservice architecture using the MQTT protocol

Summary:

In this paper, the design and implementation of a prototype system for the reliable processing of Internet of Things telemetry in an event-driven microservice architecture are described. The system connects ESP32 microcontrollers with attached DHT11 sensors, an Eclipse Mosquitto broker, and two independent microservices built with Spring Boot, which receive temperature and humidity measurements over the MQTT protocol and store them in a PostgreSQL database extended with TimescaleDB. The focus of the work is a layered reliability strategy: the QoS 1 quality of service level guarantees at-least-once delivery, while possible duplicates are removed by application-level deduplication based on a Redis store and a unique message identifier. Additional resilience to connection loss is provided by a persistent broker session and an in-memory buffer on the device. Experimental evaluation across six scenarios, including broker failure, service failure, and high load, demonstrated zero message loss, correct duplicate suppression, and a median latency below 20 ms under steady load, confirming the effectiveness of the proposed approach.

Keywords: Internet of Things, MQTT, microservices, deduplication, reliability, QoS

Sadržaj

Sažetak.....	1
Summary.....	2
Sadržaj	3
Uvod	5
1. Teorijska podloga	7
1.1. Internet stvari: pregled.....	7
1.2. Protokol MQTT	8
1.3. Mikroservisna arhitektura i arhitektura vođena događajima	11
1.4. Deduplikacija poruka i idempotentna obrada	12
1.5. Redis kao pohrana u radnoj memoriji.....	12
1.6. PostgreSQL + TimescaleDB za perzistenciju podataka	13
2. Arhitektura sustava	15
2.1. Pregled arhitekture.....	15
2.2. Tijek obrade poruka	16
2.3. Strategija deduplikacije	18
2.4. Obrasci pouzdanosti	20
2.5. Infrastrukturna topologija	21
3. Ugrađeni program uređaja ESP32	23
3.1. Sklopovska konfiguracija	23
3.2. Struktura ugrađenog programa	25
3.3. NTP sinkronizacija i identifikatori poruka	27
3.4. Objava telemetrije.....	28
3.5. Logika ponovnog spajanja i međuspremnik	30
4. Implementacija mikroservisa.....	35

4.1.	Pregled Spring Boot projekata.....	35
4.2.	MQTT konfiguracija i konekcija	36
4.3.	Servis za unos: deduplikacija i pohrana	39
4.4.	Servis za upozorenja: vrednovanje pravila i obavijesti	43
4.5.	Shema baze podataka.....	47
5.	Infrastruktura i pokretanje	50
5.1.	Docker Compose konfiguracija	50
5.2.	Konfiguracija Mosquitto posrednika	54
5.3.	Konfiguracija Redis pohrane	55
5.4.	Pokretanje i verifikacija sustava	55
6.	Testiranje i rezultati	57
6.1.	Metodologija testiranja	57
6.2.	T1: Stabilno opterećenje	59
6.3.	T2: Usporedba razina QoS	59
6.4.	T3: Obrada duplikata	61
6.5.	T4: Kvar posrednika	61
6.6.	T5: Kvar servisa za unos	62
6.7.	T6: Deduplikacija pod opterećenjem.....	63
6.8.	Analiza rezultata i zaključci testiranja	64
	Zaključak	66
	Literatura	67
	Izjava o korištenju umjetne inteligencije.....	69
	Skraćenice.....	70

Uvod

Broj uređaja Interneta stvari (engl. *Internet of Things, IoT*) u proteklom je desetljeću narastao iz milijuna u desetke milijardi, pri čemu su senzori za temperaturu, vlažnost, položaj i potrošnju energije postali sastavni dio industrijskih postrojenja, pametnih zgrada i kućanstava. Takvi uređaji kontinuirano šalju telemetrijske podatke prema poslužiteljskoj strani, gdje se podaci pohranjuju, analiziraju i koriste kao temelj za reakciju u stvarnom vremenu. Klasični model komunikacije temeljen na protokolu HTTP, oblikovan za sinkrone zahtjeve i odgovore između preglednika i poslužitelja, pokazuje znatna ograničenja u tom kontekstu: otvaranje TCP sesije po svakom mjerenju uvodi suvišan režijski trošak, sinkrona priroda zahtjeva loše podnosi prekid mreže, a tijesna sprega između proizvođača i potrošača podataka otežava skaliranje sustava.

Kao odgovor na navedena ograničenja, u domeni IoT sustava ustalio se protokol MQTT (engl. *Message Queuing Telemetry Transport*) zajedno s arhitekturom vođenom događajima (engl. *event-driven architecture*). MQTT je lagani protokol objave i pretplate (engl. *publish-subscribe*) standardiziran normom ISO/IEC 20922:2016, kod kojeg uređaji asinkrono objavljuju poruke na hijerarhijski organizirane teme, dok ih posrednik (engl. *broker*) usmjerava prema svim zainteresiranim pretplatnicima. Time se proizvođači i potrošači podataka odvajaju u prostoru i vremenu, a različite razine kvalitete usluge (engl. *Quality of Service, QoS*) omogućuju oblikovanje kompromisa između pouzdanosti dostave i utroška mrežnih sredstava.

U sklopu ovog rada osmišljen je i implementiran prototip sustava za pouzdanu obradu IoT telemetrije utemeljen na navedenim principima. Dva mikroupravljača ESP32 s pridruženim DHT11 senzorima objavljuju mjerenja temperature i vlažnosti na posrednika Eclipse Mosquitto, a dva neovisna mikroservisa razvijena u okviru Spring Boot (servis za unos i servis za upozorenja) paralelno pretplatom konzumiraju iste događaje. Servis za unos pohranjuje očitavanja u relacijsku bazu PostgreSQL proširenu ekstenzijom TimescaleDB, koja vremensko-serijske podatke organizira u hipertablice (engl. *hypertables*) i time omogućuje učinkovite upite nad velikim vremenskim rasponima. Servis za upozorenja paralelno vrednuje konfigurabilna pravila i šalje obavijesti vanjskim sustavima putem HTTP poziva. Posebna je pažnja posvećena pouzdanosti: kombinacija razine QoS 1, perzistentne sesije posrednika (`cleanSession=false`), trajne pohrane stanja klijenta i aplikacijske

deduplikacije u bazi Redis osigurava da se nijedna poruka ne izgubi niti obradi više puta unutar realnih prozora retransmisije.

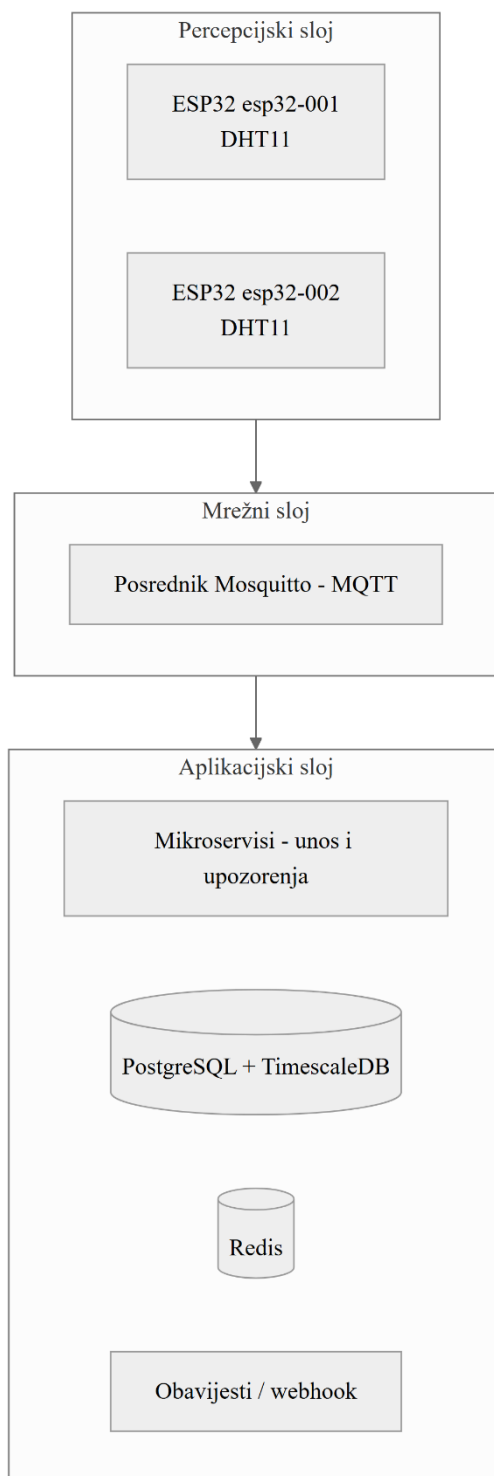
Cilj rada je pokazati da se opisanim slojevitim pristupom mogu istovremeno postići pouzdana dostava poruka od senzora do trajne pohrane, idempotentna obrada i predvidljiva latencija pod realnim opterećenjem. U nastavku se najprije izlaže teorijska podloga vezana uz IoT, MQTT, mikroservise i strategije deduplikacije (poglavlje 1), zatim se opisuje arhitektura cjelokupnog sustava (poglavlje 2), implementacija ugrađenog programa za uređaj ESP32 (poglavlje 3) te implementacija mikroservisa (poglavlje 4). Slijedi opis infrastrukture i postupka pokretanja (poglavlje 5), nakon čega se predstavljaju scenariji testiranja i izmjerene metrike pouzdanosti i performansi (poglavlje 6). Rad završava zaključkom i prijedlozima budućih nadogradnji (Zaključak).

1. Teorijska podloga

1.1. Internet stvari: pregled

Internet stvari (engl. *Internet of Things*, *IoT*) označava paradigmu u kojoj se svakodnevni fizički objekti (senzori, izvršni elementi, uređaji bijele tehnike, vozila i industrijska oprema) opremaju računalnim i komunikacijskim sposobnostima te povezuju u zajedničku mrežu radi razmjene podataka. Procjene specijaliziranih analitičkih kuća pokazuju da je broj povezanih uređaja krajem 2024. godine premašio 18 milijardi, a očekuje se da će do kraja desetljeća prijeći granicu od 30 milijardi. Takav rast otvara nove primjene u području industrije 4.0, pametnih gradova, precizne poljoprivrede i zdravstva, ali istodobno postavlja stroge zahtjeve pred poslužiteljsku infrastrukturu, koja mora pouzdano prihvatiti, pohraniti i analizirati neprekinut tok mjerenja s velikog broja izvora.

U literaturi se IoT sustavi tradicionalno prikazuju troslojnim modelom [1]. Percepcijski sloj (engl. *perception layer*) obuhvaća senzore i mikroupravljače koji prikupljaju fizikalne veličine iz okoline. Mrežni sloj (engl. *network layer*) odgovoran je za prijenos prikupljenih podataka preko bežičnih i žičnih komunikacijskih tehnologija te često uključuje posrednika poruka kao središnju točku okupljanja prometa. Aplikacijski sloj (engl. *application layer*) sadrži poslužiteljske komponente koje podatke pohranjuju, obrađuju i predstavljaju krajnjem korisniku ili drugim sustavima. Sustav opisan u ovom radu slijedi tu raščlambu: uređaji ESP32 čine percepcijski sloj, posrednik Mosquitto i protokol MQTT mrežni sloj, dok skup mikroservisa, baza podataka i sustav obavješćivanja čine aplikacijski sloj. Opisani troslojni model i smještaj komponenata ovog sustava prikazani su na slici (Sl. 1.1).



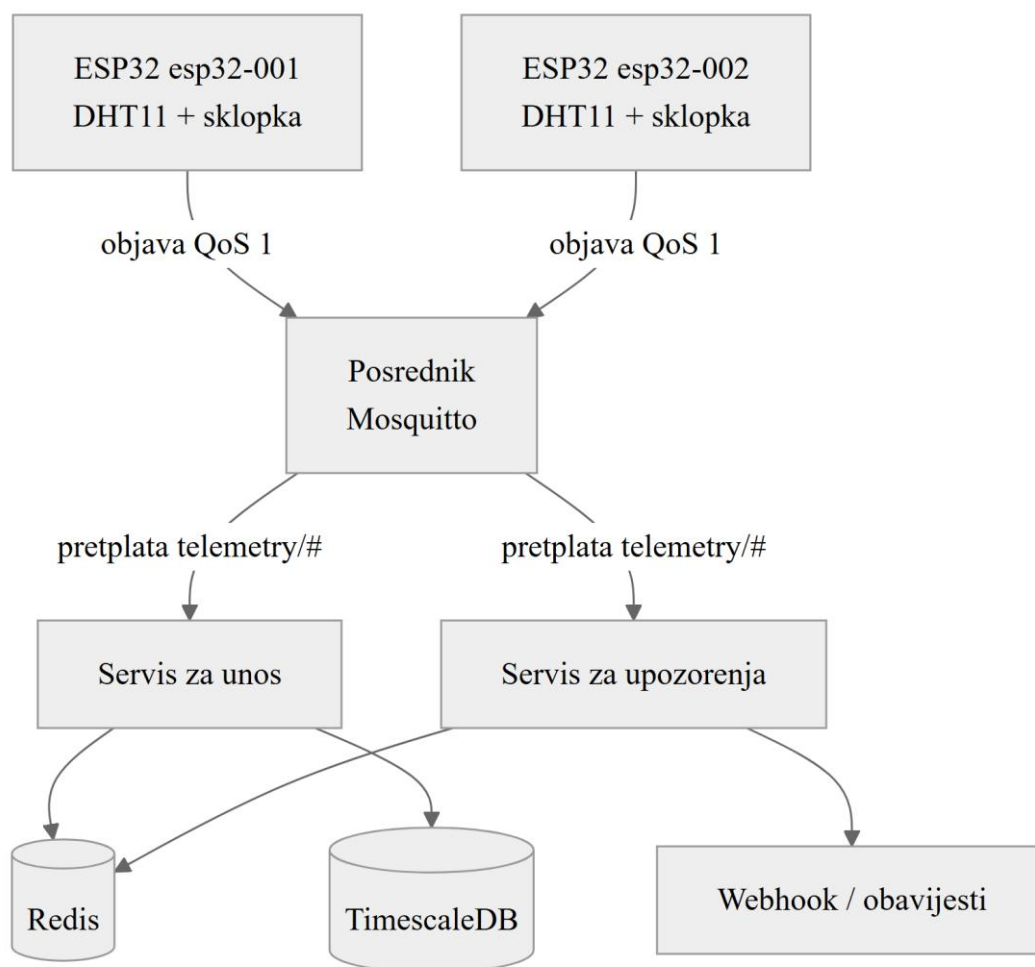
Sl. 1.1 Troslojni model IoT arhitekture.

1.2. Protokol MQTT

Protokol MQTT (engl. *Message Queuing Telemetry Transport*) razvijen je krajem 1990-ih u tvrtkama IBM i Arcom za potrebe nadzora udaljenih dijelova naftovoda, gdje su uređaji raspolagali skromnom procesorskom snagom i nepouzdanim satelitskim vezama. Iste su

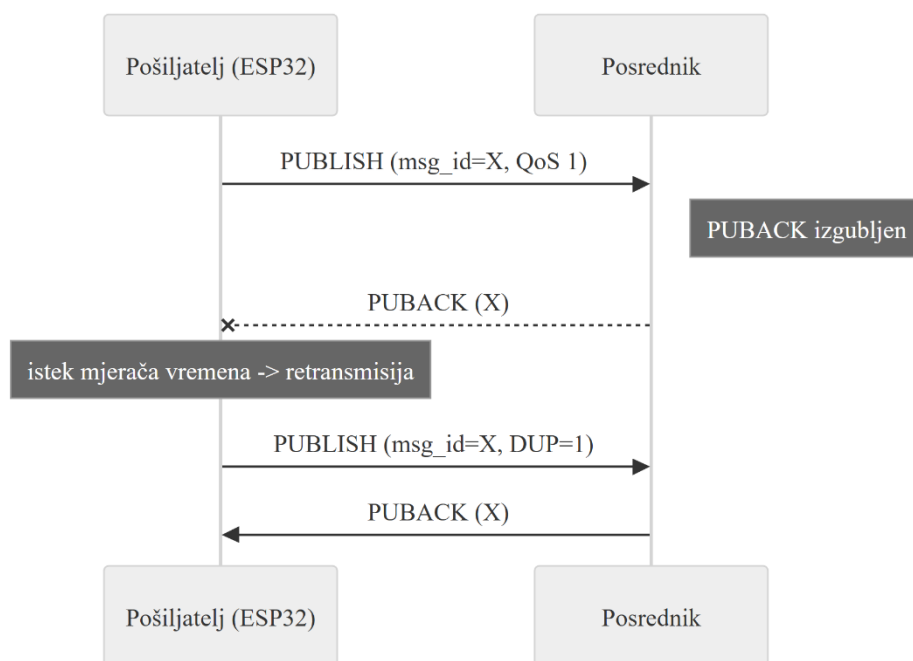
značajke, to jest male kodne staze, niska potrošnja energije i otpornost na prekide veze, kasnije proglašene poželjnima u širem području IoT primjena, što je protokol učinilo de facto standardom. Verzija 3.1.1 standardizirana je 2016. godine normom ISO/IEC 20922:2016 [2], dok je trenutno aktualna verzija 5.0 dodatno proširila funkcionalnost zaglavlja poruka i razlozima neuspjeha [3].

Komunikacijski model MQTT-a temelji se na obrascu objavi-pretplati (engl. *publish-subscribe*) s posrednikom (engl. *broker*) kao središnjom razmjenom. Klijenti se na posrednika spajaju TCP-om i nakon uspostave sesije mogu objavljevati poruke na proizvoljno odabrane hijerarhijski strukturirane teme (engl. *topics*), poput `telemetry/esp32-001/temperature`, te se pretplatiti na vlastite uzorke tema. Posrednik pri primitku poruke pronalazi sve pretplatnike čiji uzorak odgovara temi i isporučuje im kopiju, pri čemu se za podudaranje mogu koristiti zamjenski znakovi `+` (jedna razina hijerarhije) i `#` (proizvoljan broj razina na kraju uzorka). Time se izvori i potrošači podataka u potpunosti odvajaju: uređaj koji objavljuje očitavanje nema saznanja o tome koliko mikroservisa, ako uopće, te podatke konzumira. Opisana topologija objave i pretplate prikazana je na slici (Sl. 1.2).



Sl. 1.2 Topologija objave i pretplate: ESP32 uređaji, posrednik i mikroservisi.

Specifikacija definira tri razine kvalitete usluge (engl. *Quality of Service*, *QoS*) koje opisuju jamstva isporuke. Razina QoS 0 („najviše jednom“) šalje poruku bez potvrde i prikladna je za podatke kod kojih je gubitak prihvatljiv. Razina QoS 1 („barem jednom“) uvodi potvrdni paket (PUBACK) i pošiljatelj retransmitira poruku do potvrde, čime se eliminira gubitak, ali postoji mogućnost duplikata. Taj je slijed prikazan na slici (Sl. 1.3). Razina QoS 2 („točno jednom“) proširuje protokol rukovanjem u četiri koraka (PUBLISH–PUBREC–PUBREL–PUBCOMP) i u potpunosti uklanja duplikate uz najveći režijski trošak. U opisanom sustavu odabrana je razina QoS 1 kao razuman kompromis između pouzdanosti i performansi: gubitak je nedopustiv jer telemetrijska očitavanja predstavljaju osnovu za analizu i upozorenja, dok se eventualni duplikati učinkovito uklanjaju na aplikacijskom sloju opisanom u poglavlju 1.4.



Sl. 1.3 Slijed isporuke poruke pri razini QoS 1 (PUBLISH / PUBACK i retransmisija).

1.3. Mikroservisna arhitektura i arhitektura vođena događajima

Mikroservisna arhitektura (engl. *microservices architecture*) stil je oblikovanja poslužiteljskih sustava u kojem se ukupna funkcionalnost rastavlja na više malih, neovisno razvijanih i pokretanih servisa, od kojih svaki posjeduje usku poslovnu odgovornost i vlastiti spremnik podataka [4]. Time se postižu bolja podjela rada među timovima, jednostavnije neovisno skaliranje pojedinih komponenti i kraći ciklus dostave promjena u proizvodno okruženje [5]. U ovom je radu funkcionalnost cjelovitog sustava namjerno podijeljena na dva mikroservisa (servis za unos telemetrije i servis za upozorenja) jer se njihove odgovornosti i obrasci pristupa podacima jasno razlikuju.

Mikroservisi prirodno se uklapaju u arhitekturu vođenu događajima (engl. *event-driven architecture, EDA*), kod koje komponente komuniciraju asinkronom razmjenom događaja umjesto izravnim sinkronim pozivima. Posrednik poruka pritom služi kao zajednička sabirnica događaja (engl. *event bus*): proizvođač događaja ne zna tko ih konzumira, a konzumenti se mogu dodavati, uklanjati i nadograđivati bez utjecaja na ostatak sustava. U opisanom sustavu Mosquitto ima upravo tu ulogu: oba mikroservisa pretplaćuju se na uzorak `telemetry/#` i neovisno obrađuju istu pristiglu poruku, čime se postiže visoka razina razdvojenosti i jednostavno proširenje budućim potrošačima.

1.4. Deduplikacija poruka i idempotentna obrada

Razina QoS 1 jamči da nijedna poruka neće biti izgubljena, ali otvara mogućnost da se ista poruka dostavi više puta, primjerice kad mrežni paket s potvrdom (PUBACK) bude izgubljen i pošiljalac nakon isteka mjerača vremena (engl. *timer*) retransmitira originalnu poruku. Kako bi obrada na poslužiteljskoj strani ostala ispravna, potrebno je osigurati svojstvo idempotentnosti: ponovljena obrada iste poruke ne smije promijeniti krajnje stanje sustava niti pokrenuti dvostruke učinke poput dvostrukog upisa očitavanja ili dvostrukog slanja upozorenja.

U opisanom sustavu idempotentnost se postiže kombinacijom jedinstvenog identifikatora poruke i aplikacijske deduplikacije. Svaki uređaj svakoj poruci dodjeljuje identifikator oblika `msg_id = {device_id}-{epoch_ms}-{counter}`, koji je jedinstven kroz uređaje, ali i kroz poruke poslane unutar iste milisekunde. Pri obradi servis najprije provjerava postojanje identifikatora u Redis skupu već viđenih poruka. U slučaju pronalaska poruka se odbacuje, a inače se pohranjuje u relacijsku bazu i identifikator se bilježi u Redis. Treba istaknuti da TimescaleDB hipertablice ne dopuštaju jedinstvena ograničenja koja izostavljaju partijski stupac, zbog čega `msg_id` u shemi nije postavljen kao primarni niti jedinstveni ključ, već postoji isključivo kao indeks za pretraživanje. Cjelokupna odgovornost za sprječavanje duplikata time prelazi na Redis sloj. Vrijeme isteka Redis ključa odabrano je tako da pokrije sve realne prozore QoS 1 retransmisije, čime je vjerojatnost zakašnjelog duplikata koji izmiče Redis sloju u praksi zanemariva. Redoslijed koraka (najprije upis u bazu, potom oznaka u Redis) odabran je namjerno i usko je povezan s ručnim potvrđivanjem primitka opisanom u poglavlju 4: poruka se posredniku potvrđuje tek nakon oba koraka, pa pad servisa prije potvrde dovodi do ponovne isporuke i time sprječava gubitak poruke. Ako pad nastupi u uskom prozoru između samog upisa u bazu i oznake u Redisu, ponovljena isporuka neće biti prepoznata kao duplikat jer oznaka još nije zapisana, pa može proizvesti dodatni redak (baza zbog ograničenja hipertablica ne provodi jedinstvenost nad `msg_id`). Taj se rubni slučaj u poglavlju 6.8 navodi kao zanemariv u praksi.

1.5. Redis kao pohrana u radnoj memoriji

Redis je pohrana ključ-vrijednost u radnoj memoriji s podrškom za bogat skup struktura podataka i operacije složenosti $O(1)$ nad jednostavnim ključevima [6]. Ta svojstva čine ga prirodnim izborom za sloj brze deduplikacije postavljen između posrednika poruka i

relacijske baze jer se provjera postojanja identifikatora poruke izvodi za manje od jedne milisekunde, bez opterećivanja glavne pohrane. Svaki mikroservis koristi vlastiti prostor ključeva (engl. *keyspace*), odnosno `dedup:ingestion:*` i `dedup:alerting:*`, kako bi se izbjegli sudari i kako bi se omogućila zasebna analiza prometa po servisu.

Ključevima se postavlja vrijeme isteka (engl. *time-to-live*, *TTL*) od 3600 sekundi, čime je obuhvaćen tipičan prozor u kojem MQTT klijent retransmitira nepotvrđene QoS 1 poruke. Nakon isteka ključeva Redis ih automatski uklanja, čime je memorijsko zauzeće stabilno. Procjena pokazuje da pri stalnom prometu od 100 poruka u sekundi i prosječnoj duljini ključa od 40 bajtova zauzeće tijekom punog sata neće prijeći ~86 MB, što je za moderne poslužitelje zanemarivo. Budući da je deduplikacija u cijelosti prepuštena aplikacijskom sloju, TTL je namjerno odabran znatno iznad tipičnih MQTT retransmisijskih prozora (reda veličine sekundi do nekoliko minuta), čime je u praksi pokriven čitav vremenski raspon u kojem QoS 1 dostava može isporučiti duplikat.

1.6. PostgreSQL + TimescaleDB za perzistenciju podataka

Za trajnu pohranu očitavanja i događaja upozorenja odabrana je relacijska baza PostgreSQL [7]. Iako specijalizirane baze za vremenske serije, poput InfluxDB-a, nude posebno optimizirane operatore i kompresijske strategije, izbor je pao na PostgreSQL zbog izravne integracije s tehnologijom Spring Data JPA [8], snažnih jamstava transakcijske ispravnosti i mogućnosti kombiniranja vremenskih nizova s povezanim metapodacima u istoj bazi. Za potrebe učinkovitog rada s vremenskim nizovima PostgreSQL je proširen ekstenzijom TimescaleDB, koja relacijskim tablicama pridružuje sloj automatskog particioniranja po vremenu i specijalizirane operatore za agregaciju [9].

Migracijom `v1__init_timescale.sql`, koju u oba servisa izvršava alat Flyway pri pokretanju, kreira se ekstenzija TimescaleDB te se obje glavne tablice (`telemetry_readings` i `alert_events`) pretvaraju u hipertablice (engl. *hypertables*) particionirane po vremenskom stupcu. S aspekta razvojnih okvira hipertablica se ponaša kao obična relacijska tablica i pristup joj se ostvaruje istim JPA repozitorijima, dok TimescaleDB transparentno upravlja particijama i ubrzava upite nad velikim vremenskim rasponima.

Hipertablice u TimescaleDB-u nameću jedno arhitektonsko ograničenje: jedinstvena ograničenja (PRIMARY KEY ili UNIQUE) moraju uključivati particijski stupac. Budući da je u opisanom sustavu particijski stupac vremenska oznaka primitka (`received_at` u tablici

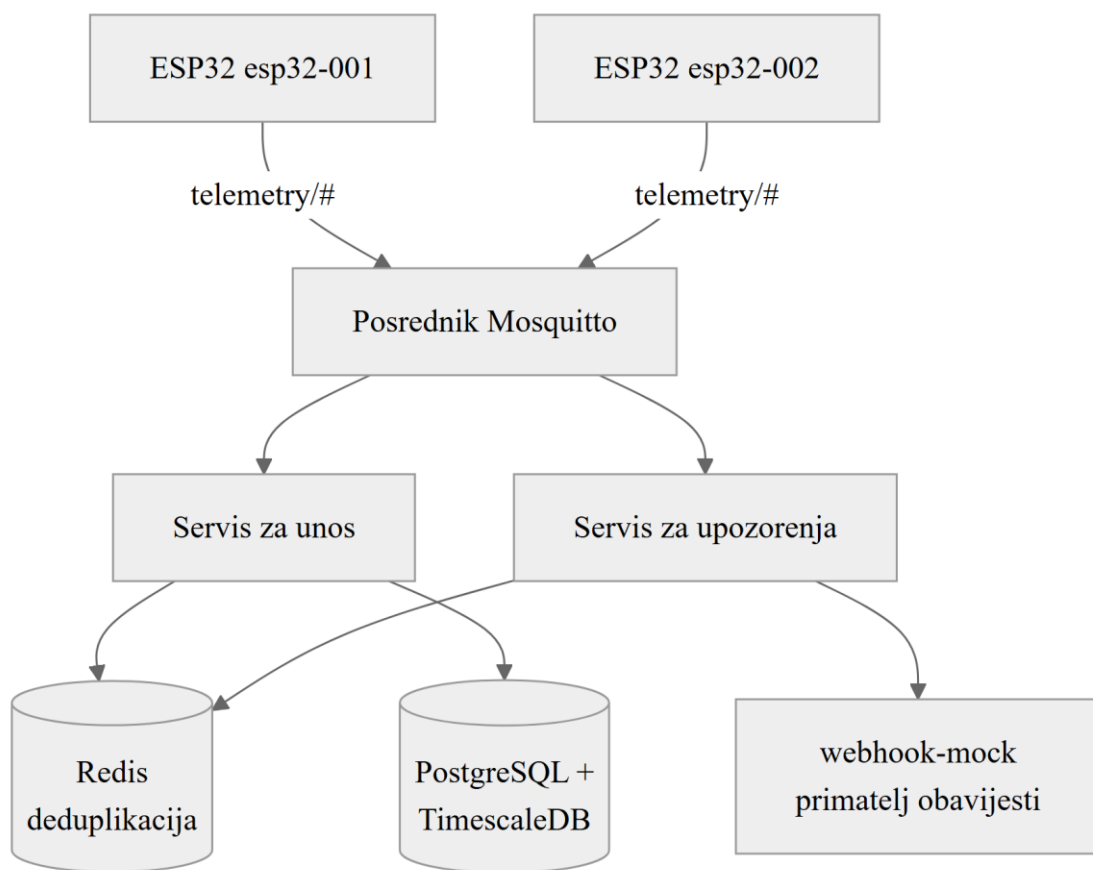
telemetry_readings, odnosno created_at u tablici alert_events), nije moguće postaviti samostalno jedinstveno ograničenje nad poljem msg_id. Stupac msg_id stoga u shemi postoji kao obični NOT NULL VARCHAR s pridruženim nejedinstvenim indeksom za brza pretraživanja, dok se idempotentnost u potpunosti osigurava aplikacijskim slojem (poglavlje 1.4). Uz polje timestamp koje dolazi iz uređaja, baza bilježi i polje received_at, koje servis za unos postavlja u trenutku obrade poruke. Razlika dvaju polja predstavlja ukupnu latenciju od trenutka mjerenja do trenutka trajne pohrane i koristi se kao temeljna mjera u eksperimentima opisanima u poglavlju 6.

2. Arhitektura sustava

2.1. Pregled arhitekture

Sustav razvijen u sklopu ovog rada sastoji se od pet logičkih komponenti koje zajedno čine cjelovit lanac pouzdane obrade IoT telemetrije: skupa od dva mikroupravljača ESP32 s pridruženim DHT11 senzorima (oznake `esp32-001` i `esp32-002`), posrednika poruka Eclipse Mosquitto, dvaju neovisnih mikroservisa razvijenih u okviru Spring Boot (servis za unos telemetrije i servis za upozorenja), pohrane Redis u radnoj memoriji za deduplikaciju te relacijske baze PostgreSQL proširene ekstenzijom TimescaleDB za trajnu pohranu očitavanja i događaja upozorenja. Uz navedene komponente, u sklopu razvojnog okruženja pokreće se i pomoćni servis koji emulira primatelja webhook poziva (engl. *webhook mock*) kako bi se mogli vjerodostojno verificirati scenariji obavješćivanja. Cjelokupna arhitektura prikazana je na slici (Sl. 2.1).

Sve komponente koje rade na poslužiteljskoj strani izvršavaju se kao zasebni Docker spremnici (engl. *containers*) povezani zajedničkom mrežom Docker Compose, što omogućuje deterministički razvojni postupak i jednostavnu zamjenu okruženja. ESP32 uređaji povezuju se na istu lokalnu mrežu putem bežične veze (Wi-Fi) i komuniciraju isključivo s posrednikom Mosquitto. Ključno arhitektonsko obilježje sustava jest da se oba mikroservisa neovisno pretplaćuju na isti uzorak tema (`telemetry/#`), čime se ostvaruje paralelna obrada istih događaja: servis za unos brine se o trajnoj pohrani očitavanja, dok servis za upozorenja istovremeno vrednuje konfigurirana pravila i šalje obavijesti vanjskim sustavima. Time se ostvaruje stroga razdvojenost odgovornosti, a buduće proširenje sustava dodatnim potrošačima (primjerice servisom za vizualizaciju u stvarnom vremenu) ne zahtijeva nikakve izmjene postojećih komponenti.



Sl. 2.1 Arhitektura sustava na visokoj razini.

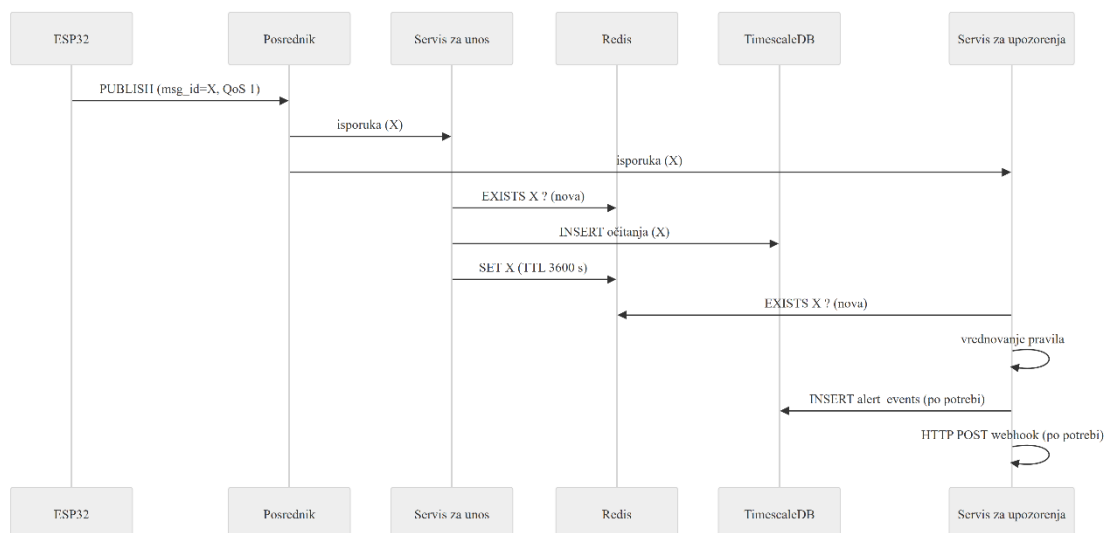
2.2. Tijek obrade poruka

Životni ciklus svake pojedine poruke započinje u petlji ugrađenog programa uređaja ESP32. Po isteku intervala objave uređaj očitava temperaturu i relativnu vlažnost s DHT11 senzora, generira jedinstveni identifikator `msg_id` opisan u poglavlju 1.4, formira JSON korisni teret (engl. *payload*) i objavljuje dvije zasebne poruke (jednu za temperaturu, drugu za vlažnost) na teme `telemetry/{DEVICE_ID}/temperature` i `telemetry/{DEVICE_ID}/humidity` s razinom QoS 1. Posrednik Mosquitto potvrđuje primitak paketom PUBACK i pohranjuje poruku u red u radnoj memoriji zajedno s metapodacima sesije, nakon čega ju isporučuje svim pretplatnicima čija pretplata odgovara temi. Cjelovit slijed obrade prikazan je na slici (Sl. 2.2).

Servis za unos po primitku poruke iz Paho klijentske biblioteke najprije deserijalizira JSON u objekt prijenosa podataka (engl. *data transfer object, DTO*), zatim provjerava postojanje vrijednosti `msg_id` u Redis skupu već viđenih poruka. Ako je identifikator već prisutan, poruka se odbacuje uz zapis informativnog događaja, čime se izbjegava dvostruka obrada.

U suprotnom se objekt mapira u JPA entitet, polje `received_at` postavlja na trenutačno vrijeme poslužitelja, a entitet se pohranjuje u tablicu `telemetry_readings` u bazi PostgreSQL. Tek nakon uspješnog upisa u bazu identifikator se bilježi u Redis s vremenom isteka od 3600 sekundi, a tek po dovršenoj obradi servis posredniku ručno potvrđuje primitak poruke (poglavlje 4.2). Ako obrada zbog prolazne pogreške ne uspije, poruka ostaje nepotvrđena i posrednik je ponovno isporučuje.

Servis za upozorenja paralelno obrađuje istu poruku po analognom obrascu, no umjesto upisa očitavanja u zajedničku tablicu telemetrije vrednuje pravila konfigurirana u datoteci `application.yml`. Ako vrijednost izlazi izvan dopuštenog raspona za odgovarajući senzor, kreira se zapis u tablici `alert_events` s pridruženom razinom ozbiljnosti (`WARNING` ili `CRITICAL`) i šalje se HTTP POST poziv prema vanjskom primatelju webhook obavijesti. Eventualne pogreške u komunikaciji s primateljem obavijesti logiraju se, ali se ne propagiraju natrag prema posredniku, čime se sprječava nepotrebna retransmisija već uspješno obrađenih poruka. Redoslijed dvaju koraka (najprije upis u bazu, potom oznaka u Redis) odabran je namjerno: ako bi servis pao u uskom intervalu između dvaju koraka, posrednik bi zbog izostanka potvrde retransmitirao poruku, a sustav bi pri sljedećoj obradi prepoznao da je zapis već prisutan u Redisu (u tom slučaju oznaka je dospjela) ili bi ga ponovno upisao kao novu pojavu, čime se ne narušava krajnja konzistentnost.

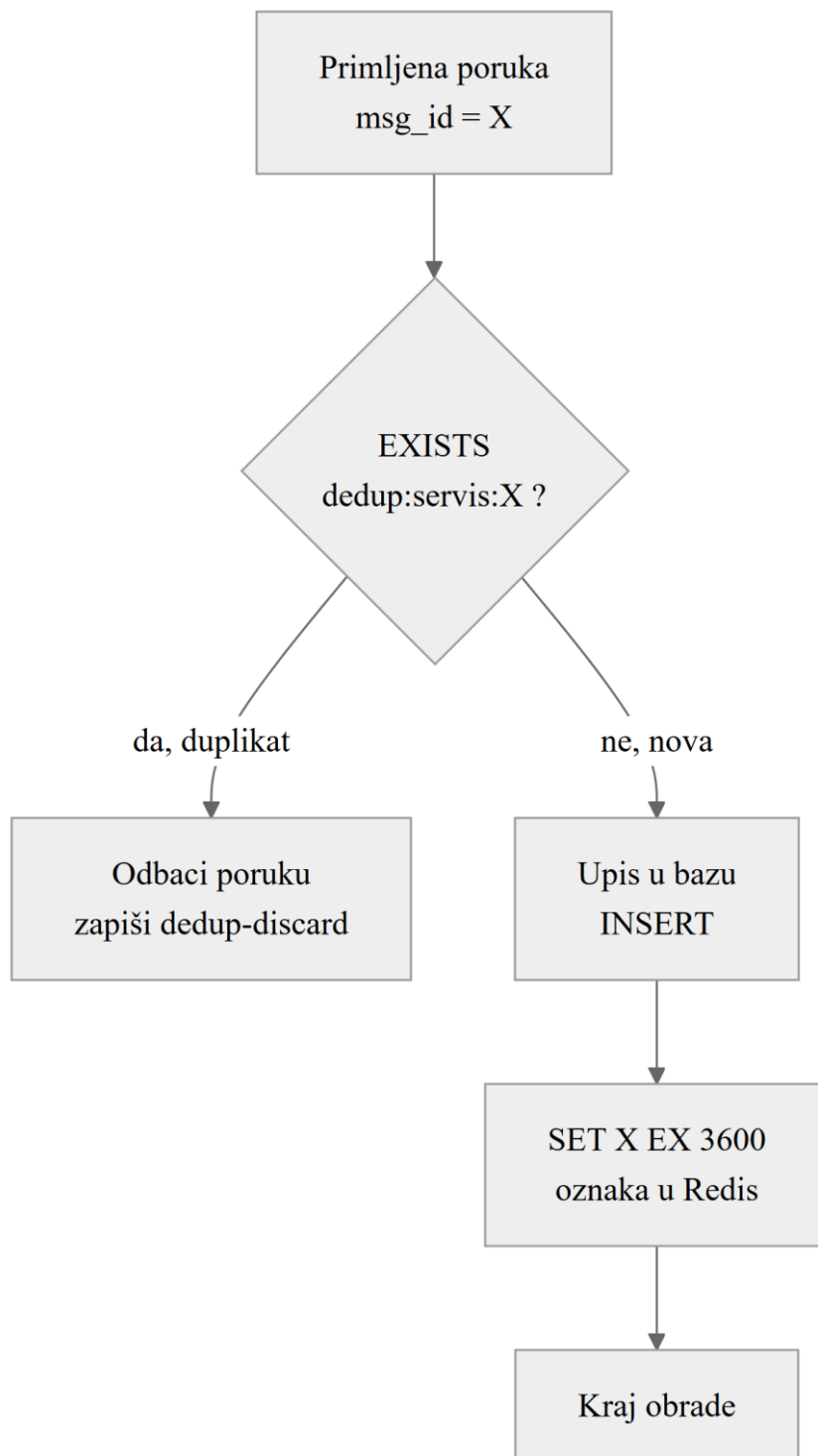


Sl. 2.2 Slijed životnog ciklusa poruke od uređaja do trajne pohrane i obavijesti.

2.3. Strategija deduplikacije

Kako je obrazloženo u poglavlju 1.4, jedinstveni identifikator `msg_id` generira se na uređaju u obliku `{DEVICE_ID}-{epoch_ms}-{counter}`, gdje je `epoch_ms` vremenska oznaka u milisekundama dobivena nakon NTP sinkronizacije, a `counter` brojač koji se inkrementira po svakoj objavi unutar iste sesije uređaja. Trodijelna struktura osigurava jedinstvenost u tri smjera: prefiks `DEVICE_ID` razlikuje izvore među različitim uređajima, polje `epoch_ms` razlikuje različite trenutke unutar istog uređaja, a brojač razlikuje pojedinačne objave izvršene unutar iste milisekunde, što je realan scenarij kada uređaj brzo objavljuje više očitavanja različitih senzora.

Aplikacijski sloj utemeljen na Redisu predstavlja središnji deduplikacijski mehanizam u sustavu. Pri svakom dolasku poruke servis poziva operaciju `EXISTS` nad ključem oblika `dedup:{servis}:{msg_id}`. Složenost te operacije jednaka je konstantnoj ($O(1)$) zbog interne implementacije Redisa kao raspršene tablice. U slučaju potvrdnog odgovora poruka se odbacuje, dok se u suprotnom obrada nastavlja, a ključ se po uspješnom upisu u bazu postavlja operacijom `SET EX 3600`. Treba istaknuti da se Paho klijent u oba servisa ponaša kao jednonitni potrošač s ograničenim pristupom jednoj poruci u trenutku, što uklanja potencijalni problem istovremenosti između provjere i oznake (engl. *time-of-check to time-of-use*, *TOCTOU*) bez potrebe za eksplicitnim zaključavanjem. Tok te odluke prikazan je na slici (Sl. 2.3).



Sl. 2.3 Tok odluke pri deduplikaciji poruke u Redis sloju.

Posebnu pažnju zaslužuju dva rubna slučaja. Prvi se odnosi na ponovno pokretanje uređaja: budući da se brojač drži u radnoj memoriji, nakon ponovnog pokretanja kreće od nule. Ako bi prvih nekoliko poruka nove sesije imalo identifikator istovjetan već poslanim porukama prethodne sesije, mogao bi se zabilježiti lažan duplikat. U praksi je taj scenarij izrazito malo vjerojatan jer se prefiks `epoch_ms` istodobno mijenja, no za potpunu otpornost u budućim

verzijama može se uvesti UUID ili broj sesije kao dodatna komponenta identifikatora. Drugi rubni slučaj odnosi se na duplikat koji posrednik isporuči nakon isteka Redis TTL prozora. Budući da je TTL od 3600 sekundi znatno veći od svih realnih MQTT retransmisijskih prozora (najviše reda veličine nekoliko minuta), takav događaj smatra se zanemarivim za praktičnu primjenu.

2.4. Obrasci pouzdanosti

Pouzdana obrada poruka u opisanom sustavu ne počiva na jednom mehanizmu, već na nekoliko međusobno komplementarnih obrazaca primijenjenih na razini klijenta, posrednika i mikroservisa. Posrednik Mosquitto pokreće se s uključenom perzistencijom (parametar `persistence true` u datoteci `mosquitto.conf`) i pohranjuje stanje sesija i neisporučenih poruka u datoteku na disku, čime se osigurava da kratkotrajno ponovo pokretanje posrednika ne dovodi do gubitka poruka koje su čekale isporuku.

Na strani mikroservisa Paho klijent [10] konfiguriran je s parametrom `cleanSession=false`, što znači da posrednik za pretplatnika čuva red poruka i nakon prekida veze. Po ponovnoj uspostavi sesije svi neisporučeni QoS 1 paketi automatski se dostavljaju. Uz to, parametar `automaticReconnect=true` omogućuje Paho klijentu da se nakon prekida sam vraća na posrednika eksponencijalno povećavajućim intervalom, bez potrebe za vanjskim nadzornim mehanizmima. Kako bi se stanje neisporučenih QoS 1 poruka sačuvalo i u slučaju iznenadnog prestanka rada mikroservisa, kao implementacija perzistencije klijenta odabran je `MqttDefaultFilePersistence` umjesto zadanog `MemoryPersistence`, čime se djelomično dovršene retransmisije nastavljaju nakon ponovnog pokretanja. Dodatno je uključeno ručno potvrđivanje primitka (engl. *manual acknowledgement*, `setManualAcks(true)`): poruka se posredniku potvrđuje tek nakon uspješne obrade, pa prolazna pogreška (primjerice privremena nedostupnost baze ili Redisa) ostavlja poruku nepotvrđenom i posrednik je ponovno isporučuje, umjesto da se ona tiho izgubi.

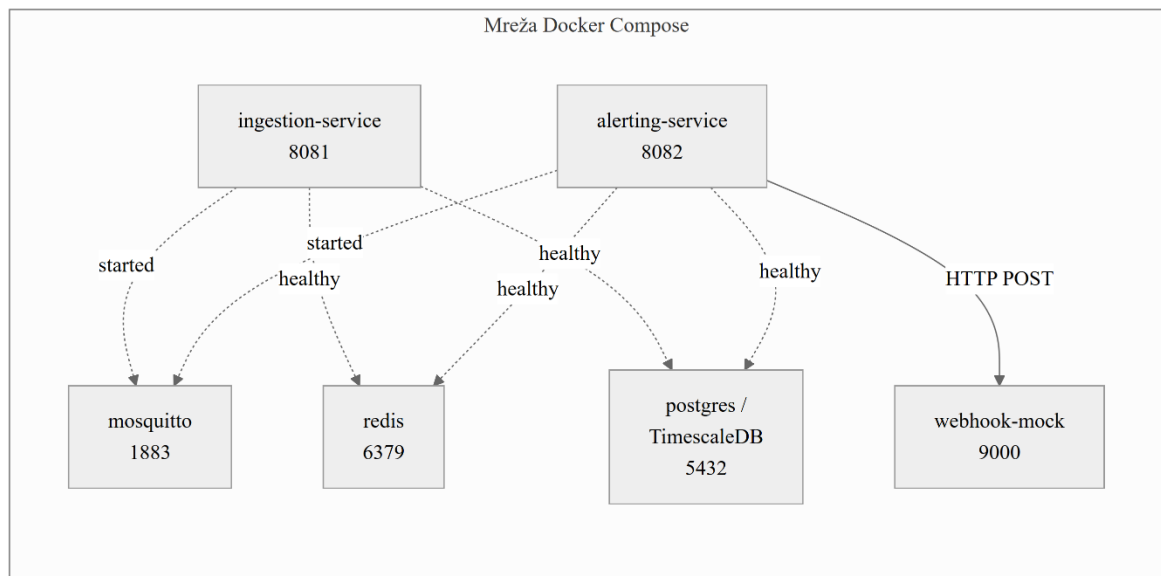
Posebno valja istaknuti povratni poziv `connectComplete`, registriran na klijentu, koji se aktivira pri svakoj uspješnoj uspostavi veze (uključujući i automatske ponovne uspostave). Pri njegovu okidanju servis ponovno poziva metodu `subscribe()` za uzorak `telemetry/#`, čime se osigurava obnova pretplate. Taj je korak nužan jer u Paho biblioteci verzije 3 pretplate nisu trajne uz `cleanSession=false` u svim scenarijima. Bez eksplicitne obnove

postojao bi rizik da mikroservis nakon prekida ostane spojen, ali bez aktivnih pretplata, što bi se očitovalo kao tihi gubitak prometa.

2.5. Infrastrukturna topologija

Cjelokupna infrastruktura opisana je deklarativno u datoteci `docker-compose.yml`, koja okuplja šest servisa: `mosquitto`, `redis`, `postgres`, `ingestion`, `alerting` i `webhook-mock`. Svaki servis pokreće se iz vlastite slike (engl. *image*) sa zadanom verzijom i izloženim ulazom prema lokalnoj mreži kako bi se omogućilo lakše otkrivanje pogrešaka tijekom razvoja. Za relacijsku bazu koristi se službena slika TimescaleDB-a koja u sebi već sadrži pripadajuću ekstenziju, dok se za posrednika koristi službena slika Eclipse Mosquitto s prilagođenom konfiguracijskom datotekom. Topologija tih servisa prikazana je na slici (Sl. 2.4).

Pravilan redoslijed pokretanja osiguran je kombinacijom mehanizama `depends_on` i `healthcheck`. Mikroservisi unos i upozorenja čekaju da posrednik, Redis i baza odgovore na zdravstvene provjere prije nego što se pokrenu, čime se izbjegavaju greške veze pri prvom pokretanju cjelokupnog okruženja. Webhook-mock servis koristi se isključivo za testne scenarije i izlaže jedan HTTP ulaz na koji servis za upozorenja šalje obavijesti. Sve primljene zahtjeve servis bilježi u standardni izlaz radi naknadne analize. Time je razvojno okruženje samodostatno: jedna naredba `docker compose up --build -d` dovoljna je za podizanje cjelokupnog sustava u stanju u kojem se mogu izvoditi sve eksperimentalne studije opisane u poglavlju 6.



Sl. 2.4 Topologija servisa u okruženju Docker Compose.

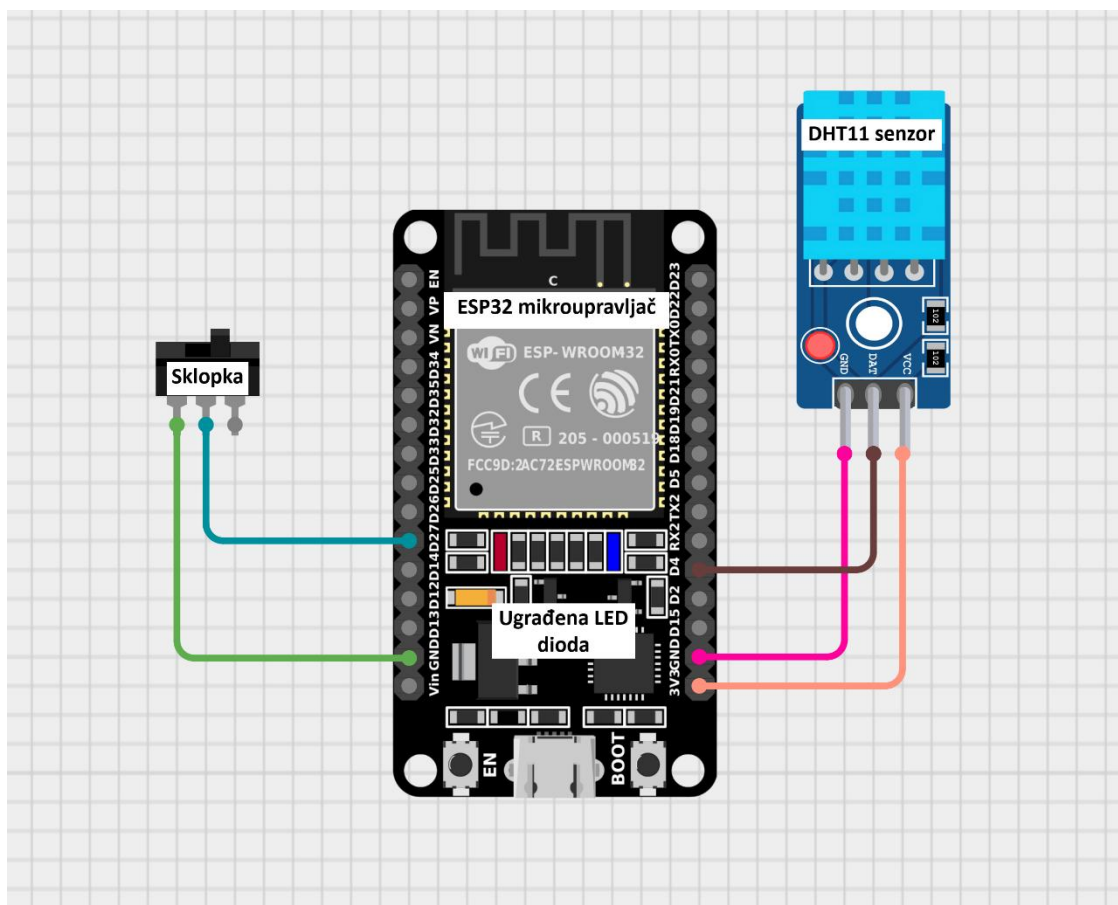
3. Ugrađeni program uređaja ESP32

Percepcijski sloj opisanog sustava čine dva mikroupravljača ESP32, od kojih svaki očitava temperaturu i relativnu vlažnost te izmjerene vrijednosti objavljuje na posrednika MQTT. Ovo poglavlje opisuje ugrađeni program (engl. *firmware*) tih uređaja: sklopovsku konfiguraciju, organizaciju izvornog koda, sinkronizaciju vremena i generiranje identifikatora poruka, postupak objave telemetrije te logiku ponovnog spajanja s pripadnim međuspremnikom (engl. *outbox*) koji osigurava pouzdanost na strani uređaja. Cijeli je program napisan u programskom jeziku C++ uz Arduino okvir [11], a obje fizičke jedinice dijele istovjetan izvorni kod, pri čemu se razlikuju isključivo u vrijednosti identifikatora uređaja.

3.1. Sklopovska konfiguracija

Svaki uređaj temelji se na razvojnoj pločici ESP32 [12] na koju je priključen senzor temperature i vlažnosti DHT11. Senzor je podatkovnom linijom spojen na nožicu `GPIO 4`, a očitava temperaturu u rasponu od 0 do 50 ° C s točnošću od oko ± 2 ° C te relativnu vlažnost u rasponu od 20 do 90 % s točnošću od oko ± 5 %, uz najveću učestalost uzorkovanja od približno jednog očitavanja u sekundi. Budući da ugrađeni program objavljuje mjerenja svakih pet sekundi, ta su ograničenja senzora s dovoljnom rezervom zadovoljena.

Za upravljanje objavom telemetrije ugrađen je prekidač (engl. *toggle switch*) spojen na nožicu `GPIO 27`. Ulaz je konfiguriran s unutarnjim pritezanjem na visoku razinu (`INPUT_PULLUP`), pa zatvaranje prekidača nožicu povlači na nisku razinu, što ugrađeni program tumači kao uključeno stanje objave. Takvim se rješenjem objava telemetrije za pojedini uređaj može ručno aktivirati i deaktivirati bez ponovnog programiranja, što je posebno korisno pri izvođenju eksperimenata opisanih u poglavlju 6. Dodatno je na nožicu `GPIO 2` spojena signalna dioda koja kratkim bljeskom potvrđuje svako uspješno očitavanje senzora. Opisani spoj prikazan je na slici (Sl. 3.1).



Sl. 3.1 Spoj razvojne pločice ESP32 i senzora DHT11 s prekidačem.

Razvoj i izgradnja ugrađenog programa ostvareni su alatom PlatformIO [13] uz platformu `espressif32` i pločicu `esp32dev`. Obje jedinice (`esp32-001` i `esp32-002`) izvršavaju identičan izvorni kod, a jedina je razlika u vrijednosti konstante `DEVICE_ID`, koja se postavlja po uređaju i ulazi u nazive MQTT tema te u identifikator poruke. Popis vanjskih biblioteka i osnovne postavke prevođenja navedeni su u datoteci `platformio.ini`, prikazanoj isječkom (Kôd 3.1).

```

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
    knolleary/PubSubClient@^2.8
    bblanchon/ArduinoJson@^7.0.0
    adafruit/DHT sensor library@^1.4.6
    adafruit/Adafruit Unified Sensor@^1.1.14

```

Kôd 3.1 Konfiguracija prevođenja u datoteci platformio.ini.

Među navedenim ovisnostima ključne su `PubSubClient` [14], koji implementira MQTT klijent prilagođen ograničenim resursima mikroupravljača, `ArduinoJson` [15] za serijalizaciju korisnog tereta u JSON oblik te `DHT sensor library` [16] za očitavanje senzora. Odabrane su lagane biblioteke male memorijske potrošnje, primjerene izvođenju na ESP32 pločici.

3.2. Struktura ugrađenog programa

Ugrađeni program je organiziran u dvije datoteke s jasno razdvojenim odgovornostima. Datoteka `config.h` okuplja sve konfiguracijske konstante (pristupne podatke za bežičnu mrežu, adresu i ulaz posrednika, identifikator uređaja, nazive tema, dodjelu nožica, interval objave, razinu QoS te parametre međuspremnik i ponovnog spajanja), dok datoteka `main.cpp` sadrži svu izvršnu logiku. Time se prilagodba pojedinog uređaja svodi na uređivanje jedne datoteke, bez zadiranja u glavni kod.

Struktura datoteke `main.cpp` slijedi uobičajeni Arduino obrazac s dvjema glavnim funkcijama. Funkcija `setup()` izvršava se jednom pri pokretanju i uspostavlja početno stanje uređaja: spajanje na bežičnu mrežu, sinkronizaciju vremena, postavljanje adrese posrednika i inicijalizaciju senzora. Funkcija `loop()` izvršava se neprekidno te objedinjuje održavanje veze, ponovno spajanje, pražnjenje međuspremnik i, kad je prekidač uključen, periodičnu objavu očitavanja. Cjelovit popis konfiguracijskih konstanti uređaja `esp32-001` prikazan je isječkom (Kôd 3.2).

Vrijednosti pristupnih podataka (`WIFI_SSID`, `WIFI_PASSWORD`, `MQTT_BROKER`) namjerno su prikazane kao rezervirana mjesta i u stvarnoj se primjeni zamjenjuju konkretnim

vrijednostima. Za uređaj `esp32-002` jedina je razlika vrijednost `DEVICE_ID` koja iznosi „`esp32-002`“.

```
#ifndef CONFIG_H
#define CONFIG_H

#define WIFI_SSID      "YOUR_WIFI_SSID"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"

#define MQTT_BROKER    "YOUR_BROKER_IP"
#define MQTT_PORT      1883
#define MQTT_USER      ""
#define MQTT_PASSWORD  ""

#define DEVICE_ID      "esp32-001"

#define TOPIC_TEMPERATURE \
    "telemetry/" DEVICE_ID "/temperature"
#define TOPIC_HUMIDITY    "telemetry/" DEVICE_ID "/humidity"

#define DHT_PIN          4
#define DHT_TYPE          DHT11

#define SWITCH_PIN       27
#define LED_PIN           2

#define PUBLISH_INTERVAL_MS 5000

#define MQTT_QOS          1

#define OUTBOX_CAPACITY    256
#define OUTBOX_DRAIN_PER_TICK 5
#define RECONNECT_INTERVAL_MS 5000

#endif
```

Kôd 3.2 Konfiguracijske konstante uređaja (`config.h`).

Nazivi tema `TOPIC_TEMPERATURE` i `TOPIC_HUMIDITY` izvedeni su nadovezivanjem niza znakova u vrijeme prevođenja, pa za uređaj `esp32-001` poprimaju vrijednosti `telemetry/esp32-001/temperature` i `telemetry/esp32-001/humidity`. Budući da i nazivi tema i identifikator poruke izravno ovise o konstanti `DEVICE_ID`, promjena te jedne

vrijednosti dostatna je za jedinstveno razlikovanje uređaja u sustavu. Konstanta `MQTT_QOS` postavljena je na 1, čime se za sve objave bira razina QoS 1 obrazložena u poglavlju 1.2.

3.3. NTP sinkronizacija i identifikatori poruka

Točnost vremenske oznake ključna je za mjerenje ukupne latencije, ali i za jedinstvenost identifikatora poruke. Stoga uređaj prije prve objave sinkronizira svoj sat s mrežnim poslužiteljima vremena pozivom `configTime()` u funkciji `setup()`, navodeći javne poslužitelje `pool.ntp.org` i `time.google.com`. Sinkronizacija je blokirajuća: ugrađeni program čeka dok ne dobije valjano vrijeme, jer bi objava prije sinkronizacije proizvela pogrešne vremenske oznake. Za razliku od funkcije `millis()`, koja mjeri samo proteklo vrijeme od pokretanja uređaja, ovdje se koristi stvarno vrijeme dobiveno funkcijom `time(nullptr)`, pomnoženo s 1000 da bi se dobile milisekunde epohe. Upravo se ta vrijednost upisuje u polje `timestamp` poruke i kasnije, na strani servisa, uspoređuje s poljem `received_at` radi izračuna latencije. Funkcija `syncNTP()` prikazana je isječkom (Kôd 3.3).

```
void syncNTP() {
    configTime(0, 0, "pool.ntp.org", "time.google.com");
    Serial.print("[NTP] Waiting for time sync");
    struct tm timeInfo;
    while (!getLocalTime(&timeInfo, 10000)) {
        Serial.print(".");
        delay(500);
    }
    Serial.printf("\n[NTP] Time synced: "
        "%04d-%02d-%02dT%02d:%02d:%02dZ\n",
        timeInfo.tm_year + 1900, timeInfo.tm_mon + 1,
        timeInfo.tm_mday,
        timeInfo.tm_hour, timeInfo.tm_min, timeInfo.tm_sec);
}
```

Kôd 3.3 Sinkronizacija vremena putem NTP-a.

Jedinstveni identifikator poruke `msg_id` generira funkcija `generateMsgId()` u obliku `{DEVICE_ID}-{epoch_ms}-{counter}`. Prvi dio (`DEVICE_ID`) razlikuje uređaje međusobno, drugi dio (`epoch_ms`) bilježi trenutak nastanka u milisekundama epohe, a treći dio je brojač `counter` koji se inkrementira pri svakoj objavi i jamči jedinstvenost čak i kad dvije poruke nastanu unutar iste milisekunde (primjerice uzastopne objave temperature i vlažnosti). Brojač se drži u radnoj memoriji (varijabla `msgCounter`) i pri ponovnom

pokretanju kreće od nule, što je obrađeno kao rubni slučaj u poglavlju 2.3. Funkcija `generateMsgId()` prikazana je isječkom (Kôd 3.4).

```
String generateMsgId() {  
    long long now_ms = (long long)time(nullptr) * 1000LL;  
    msgCounter++;  
    return String(DEVICE_ID) + "-" + String(now_ms) + "-"  
        + String(msgCounter);  
}
```

Kôd 3.4 Generiranje jedinstvenog identifikatora poruke.

3.4. Objava telemetrije

Kad je prekidač uključen, funkcija `loop()` u pravilnim razmacima (`PUBLISH_INTERVAL_MS`, zadano 5000 ms) poziva funkciju `readAndPublish()`. Ta funkcija očitava temperaturu i vlažnost sa senzora i prije objave provjerava jesu li vrijednosti valjane: ako je bilo koje očitavanje nevažeće (NaN, primjerice zbog kratkog spoja ili neispravne linije), objava se preskače i upisuje se dijagnostička poruka. U suprotnom se za svako mjerenje zasebno poziva funkcija `publishReading()`, jednom za temperaturu i jednom za vlažnost. Funkcija `readAndPublish()` prikazana je isječkom (Kôd 3.5).

```

void readAndPublish() {
    digitalWrite(LED_PIN, HIGH);

    float temperature = dht.readTemperature();
    float humidity     = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity)) {
        Serial.println(
            "[DHT] Read failed, skipping publish");
        digitalWrite(LED_PIN, LOW);
        return;
    }

    publishReading(TOPIC_TEMPERATURE, "temperature",
        temperature, "°C");
    publishReading(TOPIC_HUMIDITY, "humidity",
        humidity, "%");

    delay(100);
    digitalWrite(LED_PIN, LOW);
}

```

Kôd 3.5 Očitavanje senzora i objava telemetrije.

Funkcija `publishReading()` najprije pribavlja novi identifikator, a zatim pomoću biblioteke `ArduinoJson` sastavlja korisni teret s poljima `msg_id`, `device_id`, `sensor`, `value`, `unit` i `timestamp`. Temperatura i vlažnost objavljuju se na zasebne teme (`telemetry/{DEVICE_ID}/temperature` i `telemetry/{DEVICE_ID}/humidity`), čime se omogućuje neovisna pretplata i filtriranje po vrsti mjerenja. Objava se izvršava razinom QoS 1 samo ako je uređaj u trenutku poziva povezan. U suprotnom se poruka pohranjuje u međuspremnik opisan u sljedećem potpoglavlju, čime nijedno očitavanje ne propada zbog privremenog prekida veze. Funkcija `publishReading()` prikazana je isječkom (Kôd 3.6).

```

void publishReading(const char* topic,
                   const char* sensorType,
                   float value, const char* unit) {

    String msgId = generateMsgId();

    JsonDocument doc;
    doc["msg_id"]    = msgId;
    doc["device_id"] = DEVICE_ID;
    doc["sensor"]    = sensorType;
    doc["value"]     = value;
    doc["unit"]      = unit;
    doc["timestamp"] = (long long)time(nullptr) * 1000LL;

    char payload[256];
    serializeJson(doc, payload, sizeof(payload));

    bool ok = false;
    if (isOnline()) {
        ok = mqttClient.publish(topic, payload, MQTT_QOS);
    }

    if (ok) {
        Serial.printf("[MQTT] %s → %s (QoS %d, sent)\n",
                      topic, payload, MQTT_QOS);
    } else {
        outboxPush(topic, payload);
        Serial.printf("[Outbox] Queued (offline/"
                      "publish-fail) → %s (depth=%u)\n",
                      topic, (unsigned)outboxCount);
    }
}

```

Kôd 3.6 Sastavljanje korisnog tereta i objava očitanja.

3.5. Logika ponovnog spajanja i međuspremnik

Pri prvom pokretanju u funkciji `setup()` koriste se blokirajuće inačice spajanja `connectWiFi()` i `connectMQTT()`, koje zaustavljaju izvođenje dok se veza ne uspostavi, jer uređaj prije sinkronizacije vremena i prve objave ionako nema koristan posao. Tijekom rada,

međutim, blokiranje nije prihvatljivo jer bi zaustavilo objavu i pražnjenje međuspremnik. Zato funkcija `loop()` poziva neblokirajuće inačice `attemptWiFi()` i `attemptMQTT()`, koje pokušaj ponovnog spajanja izvode najviše jednom u intervalu `RECONNECT_INTERVAL_MS` (zadano 5000 ms) i odmah vraćaju kontrolu glavnoj petlji. Neblokirajuće funkcije ponovnog spajanja prikazane su isječkom (Kôd 3.7).

```
void attemptWiFi() {
    if (WiFi.status() == WL_CONNECTED) return;
    unsigned long now = millis();
    if (now - lastWifiAttemptMs < RECONNECT_INTERVAL_MS)
        return;
    lastWifiAttemptMs = now;

    Serial.printf("[WiFi] Reconnect attempt to %s...\n",
        WIFI_SSID);
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

void attemptMQTT() {
    if (WiFi.status() != WL_CONNECTED) return;
    if (mqttClient.connected()) return;
    unsigned long now = millis();
    if (now - lastMqttAttemptMs < RECONNECT_INTERVAL_MS)
        return;
    lastMqttAttemptMs = now;

    Serial.print("[MQTT] Reconnect attempt... ");
    bool connected = (strlen(MQTT_USER) > 0)
        ? mqttClient.connect(DEVICE_ID, MQTT_USER,
            MQTT_PASSWORD)
        : mqttClient.connect(DEVICE_ID);
    Serial.printf("%s (rc=%d)\n", connected ? "connected!"
        : "failed", mqttClient.state());
}
```

Kôd 3.7 Neblokirajuće ponovno spajanje na mrežu i posrednika.

Za razliku od Paho klijenta u mikroservisima, biblioteka `PubSubClient` na uređaju ne čuva stanje neisporučenih QoS 1 poruka između sesija, pa se pouzdanost na strani uređaja ne može osloniti na protokolni mehanizam za neisporučene poruke. Umjesto toga ugrađeni program

uvodi vlastiti međuspremnik (engl. *outbox*) izveden kao kružni spremnik (engl. *ring buffer*) fiksnog kapaciteta `OUTBOX_CAPACITY` (zadano 256 unosa). Kad uređaj nije povezan ili objava ne uspije, funkcija `outboxPush()` pohranjuje poruku u spremnik. Ako je spremnik pun, najstariji se unos odbacuje (strategija *drop-oldest*) uz zapis `[Outbox] FULL`, čime se sprječava prelijevanje memorije po cijenu gubitka najstarijih očitavanja, što je svjestan kompromis vrednovan u scenariju T4 (poglavlje 6).

Pri svakom prolasku kroz `loop()` funkcija `drainOutbox()` pokušava isprazniti nakupljene poruke, ali najviše `OUTBOX_DRAIN_PER_TICK` (zadano 5) po prolasku, kako objava akumuliranih poruka ne bi zagušila petlju nakon dužeg prekida. Pražnjenje se izvodi samo dok je uređaj povezan (`isOnline()`), a u trenutku ponovne uspostave veze nakupljena se očitavanja redom reproduciraju prema posredniku. Time se prekid veze prema posredniku premošćuje na razini uređaja, neovisno o ograničenjima biblioteke `PubSubClient`. Funkcije međuspremnika prikazane su isječkom (Kôd 3.8).

```

void outboxPush(const char* topic, const char* payload) {
    if (outboxCount == OUTBOX_CAPACITY) {
        outboxHead = (outboxHead + 1) % OUTBOX_CAPACITY;
        outboxCount--;
        Serial.println(
            "[Outbox] FULL – dropping oldest entry");
    }
    outbox[outboxTail].topic = topic;
    strncpy(outbox[outboxTail].payload, payload,
        sizeof(outbox[outboxTail].payload));
    outboxTail = (outboxTail + 1) % OUTBOX_CAPACITY;
    outboxCount++;
}

void drainOutbox() {
    if (!isOnline() || outboxCount == 0) return;

    int sent = 0;
    while (sent < OUTBOX_DRAIN_PER_TICK &&
        outboxCount > 0 && isOnline()) {
        OutboxEntry& e = outbox[outboxHead];
        if (!mqttClient.publish(e.topic, e.payload,
            MQTT_QOS)) {
            break;
        }
        Serial.printf(
            "[Outbox] Replayed → %s (remaining=%u)\n",
                e.topic, (unsigned) (outboxCount - 1));
        outboxHead = (outboxHead + 1) % OUTBOX_CAPACITY;
        outboxCount--;
        sent++;
    }
}

```

Kôd 3.8 Kružni međuspremnik i pražnjenje pri ponovnom spajanju.

Opisani slojevi pouzdanosti na strani uređaja (blokirajuće početno spajanje, neblokirajuće ponovno spajanje s ograničenim intervalom pokušaja te kružni međuspremnik s reprodukcijom pri ponovnom spajanju) zajedno s perzistentnom sesijom posrednika (`cleanSession=false`) opisanom u poglavlju 2.4 čine cjelovit lanac koji očuvanje poruka

osigurava od trenutka mjerenja do njihove isporuke posredniku. Učinkovitost tog lanca eksperimentalno se vrednuje u scenariju kvara posrednika opisanom u poglavlju 6.

4. Implementacija mikroservisa

Aplikacijski sloj sustava čine dva neovisna mikroservisa razvijena u okviru Spring Boot: servis za unos telemetrije i servis za upozorenja. Oba se servisa pretplaćuju na isti uzorak MQTT tema i neovisno obrađuju svaku pristiglu poruku, ali se razlikuju po odgovornosti: prvi trajno pohranjuje očitavanja, a drugi vrednuje pravila i šalje obavijesti. Ovo poglavlje opisuje njihovu zajedničku tehnološku osnovu, konfiguraciju MQTT veze, mehanizam deduplikacije, logiku svakog servisa te shemu baze podataka, uz isječke ključnih dijelova izvornog koda.

4.1. Pregled Spring Boot projekata

Oba su mikroservisa zasebni Maven projekti utemeljeni na verziji Spring Boot 3.2.5 uz Javu 17 [17], a dijele istovjetan skup ovisnosti. Za pristup relacijskoj bazi koristi se `spring-boot-starter-data-jpa`, za rad s pohranom u radnoj memoriji `spring-boot-starter-data-redis`, za HTTP komunikaciju i ugrađeni poslužitelj `spring-boot-starter-web`, dok MQTT vezu ostvaruje klijentska biblioteka `org.eclipse.paho.client.mqttv3`. Migracijama sheme upravlja `flyway-core`, a biblioteka `lombok` uklanja ponavljajući kod (engl. *boilerplate*) automatskim generiranjem metoda pristupa, konstruktora i sličnog.

Unutar svakog projekta kod je organiziran u pakete prema odgovornosti: `config` sadrži konfiguracijske razrede (MQTT klijent, pravila upozorenja), `dto` objekte za prijenos podataka u koje se deserijaliziraju MQTT poruke, `entity` JPA entitete preslikane na tablice baze, `repository` Spring Data repozitorije te `service` poslovnu logiku obrade poruka, deduplikacije i obavješćivanja. Takva podjela odražava tijek obrade poruke i olakšava neovisno održavanje pojedinih dijelova.

Sve okolinski ovisne postavke (adresa baze, posrednika i Redisa) navedene su u datoteci `application.yml` s mogućnošću nadjačavanja putem varijabli okoline, pri čemu su zadane vrijednosti prikladne za lokalni razvoj, a u kontejnerskom okruženju nadjačavaju se vrijednostima koje propisuje `docker compose`. Time se ista slika servisa može pokrenuti i lokalno i u kontejneru bez izmjene koda, kako prikazuje sljedeći isječak datoteke `application.yml` servisa za unos.

Izvadak iz datoteke `application.yml` (servis za unos) prikazan je isječkom (Kôd 4.1):

```

spring:
  datasource:
    url: ${SPRING_DATASOURCE_URL:}
    jdbc:postgresql://localhost:5432/iot_telemetry
    username: ${SPRING_DATASOURCE_USERNAME:postgres}
    password: ${SPRING_DATASOURCE_PASSWORD:postgres}
  data:
    redis:
      host: ${SPRING_DATA_REDIS_HOST:localhost}
      port: ${SPRING_DATA_REDIS_PORT:6379}

mqtt:
  broker: ${MQTT_BROKER:tcp://localhost:1883}
  client-id: ingestion-service
  topic: "telemetry/#"
  qos: 1

deduplication:
  ttl-seconds: 3600

```

Kôd 4.1 Izvadak konfiguracije servisa za unos (application.yml).

Zapis oblika `${VARIJABLA:zadano}` označava vrijednost iz varijable okoline s pričuvnom zadanom vrijednošću. Jedina bitna razlika u konfiguraciji dvaju servisa jest vrijednost `client-id` (`ingestion-service` odnosno `alerting-service`), čime svaki servis posredniku predstavlja zaseban i postojan identitet sesije, što je preduvjet za rad perzistentne sesije opisane u poglavlju 2.4.

4.2. MQTT konfiguracija i konekcija

MQTT vezu u oba servisa uspostavlja razred `MqttConfig`, koji ujedno implementira sučelje `MqttCallbackExtended` te tako objedinjuje konfiguraciju klijenta i obradu povratnih poziva. Pri dolasku poruke metoda `messageArrived()` prosljeđuje korisni teret odgovarajućem servisu pozivom `processMessage()`, a tek ako obrada uspije, posredniku ručno potvrđuje primitak. Kako bi se izbjegla kružna ovisnost između konfiguracije i servisa, servis se ubrizgava s anotacijom `@Lazy`.

Pri stvaranju klijenta odabire se `MqttDefaultFilePersistence` umjesto zadanog `MemoryPersistence`, čime se stanje neisporučenih QoS 1 poruka čuva na disku i preživljava

ponovno pokretanje servisa. Uz to se pozivom `setManualAcks(true)` isključuje automatsko potvrđivanje primitka koje Paho inače obavlja po povratku iz metode `messageArrived()`. Poruka se posredniku potvrđuje tek nakon što je servis uspješno obradi (poglavlje 4.3). Opcije povezivanja postavljaju se na `setCleanSession(false)` (posrednik čuva red poruka za pretplatnika i nakon prekida veze) i `setAutomaticReconnect(true)` (Paho klijent se nakon prekida sam vraća na posrednika). Time se na razini biblioteke ostvaruju obrasci pouzdanosti obrazloženi u poglavlju 2.4.

Ključni dio razreda `MqttConfig` servisa za unos (radi preglednosti izostavljene su deklaracije paketa i uvoza te ostali povratni pozivi) prikazan je isječkom (Kôd 4.2):

```

@PostConstruct
public void connect() throws MqttException {
    mqttClient = new MqttClient(broker, clientId,
                               new MqttDefaultFilePersistence(
                                   "/tmp/mqtt-"
                                   + clientId));
    mqttClient.setManualAcks(true);
    mqttClient.setCallback(this);

    MqttConnectOptions options = new MqttConnectOptions();
    options.setCleanSession(false);
    options.setAutomaticReconnect(true);

    mqttClient.connect(options);
}

@Override
public void connectComplete(boolean reconnect,
                             String serverURI) {
    log.info("Connected to MQTT broker: {} (reconnect={})",
            serverURI, reconnect);
    Thread subscribeThread = new Thread(() -> {
        try {
            mqttClient.subscribe(topic, qos);
            log.info("Subscribed to topic: {} with QoS {}",
                    topic, qos);
        } catch (MqttException e) {
            log.error("Failed to subscribe to topic: {}",
                    topic, e);
        }
    }, "mqtt-subscribe");
    subscribeThread.setDaemon(true);
    subscribeThread.start();
}

@Override
public void messageArrived(String topic,
                            MqttMessage message) {
    String payload = new String(message.getPayload());
    if (ingestionService.processMessage(topic, payload)) {
        try {

```

```

        mqttClient.messageArrivedComplete(
            message.getId(),
            message.getQos());
    } catch (MqttException e) {
        log.error("Failed to acknowledge message id={}",
            message.getId(), e);
    }
}
}

```

Kôd 4.2 Uspostava MQTT veze i obnova pretplate.

Povratni poziv `connectComplete` aktivira se pri svakoj uspješnoj uspostavi veze, uključujući automatske ponovne uspostave, i u njemu se ponovno poziva `subscribe()` za uzorak `telemetry/#`. Poziv se izvršava u zasebnoj dretvi (`mqtt-subscribe`) jer `connectComplete` teče na internoj dretvi povratnih poziva Paho klijenta. Izravno pozivanje `subscribe()` na toj dretvi moglo bi blokirati obradu veze i u nekim slučajevima dovesti do zastoja, pa se obnova pretplate namjerno postavlja izvan nje. Time se pretplata pouzdano obnavlja nakon svakog prekida, što je nužno jer Paho klijent verzije 3 u nekim scenarijima ne zadržava pretplate, pa bi bez ove obnove servis ostao spojen, ali bez aktivnih pretplata, što bi se očitovalo kao tihi gubitak prometa. Razred servisa za upozorenja gotovo je istovjetan i razlikuje se samo po servisu kojem prosljeđuje poruku.

4.3. Servis za unos: deduplikacija i pohrana

Deduplikaciju provodi razred `DeduplicationService`, tanak omotač oko Redis klijenta `StringRedisTemplate`. Metoda `isDuplicate()` provjerava postojanje ključa (operacija `hasKey`), a metoda `markProcessed()` zapisuje ključ s vremenom isteka jednakim konfiguriranom TTL-u (3600 sekundi). Svi ključevi nose prefiks `dedup:ingestion:`, čime su odvojeni od ključeva servisa za upozorenja i omogućuju zasebnu analizu prometa. Razred `DeduplicationService` prikazan je isječkom (Kôd 4.3).


```

@Service
public class DeduplicationService {

    private static final String KEY_PREFIX =
        "dedup:ingestion:";

    private final StringRedisTemplate redisTemplate;
    private final long ttlSeconds;

    public DeduplicationService(
        StringRedisTemplate redisTemplate,
        @Value("${deduplication.ttl-seconds}")
        long ttlSeconds) {
        this.redisTemplate = redisTemplate;
        this.ttlSeconds = ttlSeconds;
    }

    public boolean isDuplicate(String msgId) {
        return Boolean.TRUE.equals(redisTemplate.hasKey(
            KEY_PREFIX + msgId));
    }

    public void markProcessed(String msgId) {
        redisTemplate.opsForValue().set(
            KEY_PREFIX + msgId, "1",
            Duration.ofSeconds(ttlSeconds));
    }
}

```

Kôd 4.3 Servis za deduplikaciju nad Redisom.

Glavnu logiku objedinjuje metoda `processMessage()`. Korisni teret najprije se deserijalizira u objekt `TelemetryMessage`, pri čemu anotacija `@JsonProperty` preslikava polja JSON-a u zmijskom zapisu (npr. `msg_id`) na polja u devinom zapisu (npr. `msgId`). Slijedi provjera duplikata: ako je identifikator već viđen, poruka se uz informativni zapis odbacuje. U suprotnom se podaci preslikavaju u JPA entitet `TelemetryReading`, polje `received_at` postavlja se na trenutačno vrijeme poslužitelja (`Instant.now()`), entitet se pohranjuje pozivom `save()`, i tek nakon uspješnog upisa identifikator se bilježi u Redis pozivom `markProcessed()`. Metoda vraća vrijednost tipa `boolean` kojom javlja treba li poruku potvrditi posredniku: `true` za potvrdu (uspješna obrada ili neispravna poruka koju

nema smisla ponavljati), a `false` za prolaznu pogrešku zbog koje poruka ostaje nepotvrđena i biva ponovno isporučena. Metoda `processMessage()` prikazana je isječkom (Kôd 4.4).

```

public boolean processMessage(String topic, String payload) {
    TelemetryMessage message;
    try {
        message = objectMapper.readValue(payload,
            TelemetryMessage.class);
    } catch (Exception e) {
        log.error("[Ingestion] Discarding unparseable "
            + "message from topic={}: {}",
            topic, e.getMessage());
        return true;
    }

    try {
        if (deduplicationService.isDuplicate(
            message.getMsgId())) {
            log.info(
                "[Dedup] Duplicate msg_id={}, discarding",
                message.getMsgId());
            return true;
        }

        TelemetryReading reading = new TelemetryReading();
        reading.setMsgId(message.getMsgId());
        reading.setDeviceId(message.getDeviceId());
        reading.setSensor(message.getSensor());
        reading.setValue(message.getValue());
        reading.setUnit(message.getUnit());
        reading.setTimestamp(message.getTimestamp());
        reading.setReceivedAt(Instant.now());

        telemetryRepository.save(reading);
        deduplicationService.markProcessed(
            message.getMsgId());

        log.info("[Ingestion] Stored msg_id={} "
            + "device={} sensor={} value={}",
            message.getMsgId(), message.getDeviceId(),
            message.getSensor(), message.getValue());
        return true;
    } catch (Exception e) {

```

```

        log.error(
            "[Ingestion] Transient failure for msg_id={}, "
            + "leaving unacknowledged for redelivery: {}",
            message.getMsgId(), e.getMessage());
        return false;
    }
}

```

Kôd 4.4 Obrada poruke: deduplikacija i pohrana.

Obrada je podijeljena u dva odsječka. Neuspjela deserijalizacija (primjerice neispravan JSON) tretira se kao trajno neispravna poruka: pogreška se zapisuje, a metoda vraća `true`, čime se takva poruka potvrđuje i odbacuje umjesto da je posrednik beskonačno ponovno isporučuje. Drugu cjelinu, samu obradu, obavlja zaseban blok `try/catch`. Ako u njoj dođe do prolazne pogreške (primjerice privremena nedostupnost baze ili Redisa), metoda vraća `false` i poruka ostaje nepotvrđena, pa je posrednik kasnije ponovno isporučuje. Tek po uspješnoj obradi metoda vraća `true` i tada se izvodi ručno potvrđivanje primitka (`messageArrivedComplete`) opisano u poglavlju 4.2. Redoslijed koraka, najprije upis u bazu pa tek onda oznaka u Redis, odabran je namjerno: budući da se potvrda posredniku šalje tek nakon oba koraka, pad servisa prije potvrde dovodi do ponovne isporuke i time sprječava gubitak poruke. Ako pad nastupi u uskom prozoru između upisa u bazu i oznake u Redisu, ponovljena isporuka neće biti prepoznata kao duplikat (oznaka još nije zapisana), pa će rezultirati dodatnim retkom jer baza zbog ograničenja hipertablica ne provodi jedinstvenost nad `msg_id`. Taj se rubni slučaj u poglavlju 6.8 navodi kao zanemariv u praksi. Ručno potvrđivanje time uklanja gubitak poruka pri prolaznim pogreškama, ali ne uklanja navedeni uski prozor mogućeg dvostrukog upisa.

4.4. Servis za upozorenja: vrednovanje pravila i obavijesti

Servis za upozorenja koristi istovjetan mehanizam deduplikacije, ali s odvojenim prefiksom ključeva `dedup:alerting:`, tako da dva servisa istu poruku dedupliciraju neovisno. Kao i servis za unos, njegova metoda `processMessage()` vraća `boolean` kojim upravlja ručno potvrđivanje primitka: neispravna poruka i uspješno obrađena poruka potvrđuju se (vraćaju `true`), dok prolazna pogreška pri obradi ostavlja poruku nepotvrđenom za ponovnu isporuku. Neuspjeh slanja webhook obavijesti pritom se hvata unutar obrade i ne uzrokuje ponovnu isporuku, jer je poruka u tom trenutku već pohranjena. Pravila vrednovanja

učitavaju se iz datoteke `application.yml` u razred `AlertRulesConfig` putem anotacije `@ConfigurationProperties`. Svako pravilo određeno je nazivom senzora (`sensor`) te donjom i gornjom granicom dopuštenog raspona (`min`, `max`).

Konfiguracija pravila u datoteci `application.yml` prikazana je isječkom (Kôd 4.5):

```
alerting:
  webhook-url:
    ${ALERTING_WEBHOOK_URL:http://localhost:9000/alert}
  rules:
    - sensor: temperature
      min: 0.0
      max: 35.0
    - sensor: humidity
      min: 20.0
      max: 80.0
    - sensor: pressure
      min: 950.0
      max: 1050.0
```

Kôd 4.5 Konfiguracija pravila upozorenja.

Pravila se prema senzoru pridružuju dolaznoj poruci, pa se u ovoj postavi vrednuju očitavanja temperature i vlažnosti, dok pravilo za `pressure` ostaje pripremljeno za buduće senzore i ne utječe na trenutni rad. Ako za senzor ne postoji pravilo, poruka se samo označava obrađenom (`markProcessed()`) bez daljnje akcije. Ako vrijednost izlazi izvan dopuštenog raspona, razina ozbiljnosti određuje se metodom `determineSeverity()`.

Metoda `determineSeverity()` prag računa kao 10 % širine dopuštenog raspona (razlika gornje i donje granice). Ako vrijednost izlazi izvan raspona, ali ostaje unutar tog praga, događaj se ocjenjuje razinom `WARNING`. Ako prelazi i prošireni raspon (donja granica umanjena ili gornja uvećana za prag), dodjeljuje se razina `CRITICAL`. Time se blaga i ozbiljna odstupanja razlikuju bez uvođenja zasebnih konfiguracijskih vrijednosti. Metoda `determineSeverity()` prikazana je isječkom (Kôd 4.6).

```

private String determineSeverity(double value,
                                AlertRule rule) {
    double range = rule.getMax() - rule.getMin();
    double threshold = range * 0.1;

    if (value < rule.getMin() - threshold ||
        value > rule.getMax() + threshold) {
        return "CRITICAL";
    }
    return "WARNING";
}

```

Kôd 4.6 Određivanje razine ozbiljnosti upozorenja.

Slanje obavijesti obavlja razred `NotificationService` metodom `fireAlert()`. Ona najprije pohranjuje entitet `AlertEvent` u tablicu `alert_events` (s nazivom prekršenog pravila i razinom ozbiljnosti), a zatim, ako je konfiguriran `webhook-url`, šalje HTTP POST poziv vanjskom primatelju putem klijenta `RestTemplate`. Ključno je da se eventualne pogreške pri slanju webhook poziva samo zapisuju u zapisnik i ne propagiraju dalje: poruka je u tom trenutku već uspješno obrađena i pohranjena, pa neuspjeh vanjske obavijesti ne smije izazvati retransmisiju niti ponovnu obradu. Metoda `fireAlert()` prikazana je isječkom (Kôd 4.7).

```

public void fireAlert(TelemetryMessage msg, AlertRule rule,
    String severity) {
    String ruleName = msg.getValue() < rule.getMin()
        ? msg.getSensor() + " < " + rule.getMin()
        : msg.getSensor() + " > " + rule.getMax();

    AlertEvent event = new AlertEvent();
    event.setMsgId(msg.getMsgId());
    event.setDeviceId(msg.getDeviceId());
    event.setSensor(msg.getSensor());
    event.setValue(msg.getValue());
    event.setRuleName(ruleName);
    event.setSeverity(severity);
    event.setCreatedAt(Instant.now());
    alertEventRepository.save(event);

    if (webhookUrl != null && !webhookUrl.isBlank()) {
        try {
            Map<String, Object> body = new LinkedHashMap<>();
            body.put("msg_id", msg.getMsgId());
            body.put("device_id", msg.getDeviceId());
            body.put("sensor", msg.getSensor());
            body.put("value", msg.getValue());
            body.put("rule", ruleName);
            body.put("severity", severity);
            body.put("triggered_at",
                Instant.now().toString());

            restTemplate.postForEntity(webhookUrl, body,
                String.class);
            log.info("[Webhook] Alert posted to {}",
                webhookUrl);
        } catch (Exception e) {
            log.error("[Webhook] Failed to POST alert: {}",
                e.getMessage());
        }
    }
}

```

Kôd 4.7 Pohrana događaja upozorenja i webhook obavijest.

4.5. Shema baze podataka

Shemu baze čine dvije glavne tablice. Tablica `telemetry_readings` pohranjuje pojedinačna očitavanja i sadrži polja `msg_id`, `device_id`, `sensor`, `value`, `unit`, `timestamp` (vrijeme mjerenja s uređaja, cijeli broj milisekundi epohe) te `received_at` (vrijeme primitka na poslužitelju, koje je ujedno particijski stupac). Tablica `alert_events` bilježi okinuta upozorenja i uz analogna polja sadrži surogatni `id` tipa `BIGSERIAL` te particijski stupac `created_at`.

Obje su tablice migracijom pretvorene u TimescaleDB hipertablice particionirane po vremenu (`received_at` odnosno `created_at`) pozivom `create_hypertable`. Zbog ograničenja hipertablica, koja jedinstvena ograničenja dopuštaju samo ako uključuju particijski stupac, polje `msg_id` nije primarni niti jedinstveni ključ, već obični `NOT NULL` stupac s pridruženim nejedinstvenim indeksom (`idx_*_msg_id`) za pretragu. Uz to su postavljeni indeksi po uređaju i senzoru, odnosno razini ozbiljnosti, u kombinaciji s vremenom, radi ubrzanja tipičnih upita po vremenskim rasponima.

Dio migracije `v1__init_timescale.sql` za tablicu `telemetry_readings` prikazan je isječkom (Kôd 4.8):


```

CREATE EXTENSION IF NOT EXISTS timescaledb;

CREATE TABLE IF NOT EXISTS telemetry_readings (
    msg_id          VARCHAR(128)      NOT NULL,
    device_id       VARCHAR(64)       NOT NULL,
    sensor          VARCHAR(64)       NOT NULL,
    value           DOUBLE PRECISION  NOT NULL,
    unit            VARCHAR(16),
    "timestamp"     BIGINT,
    received_at     TIMESTAMPTZ       NOT NULL
);

SELECT create_hypertable(
    'telemetry_readings',
    'received_at',
    if_not_exists => TRUE
);

CREATE INDEX IF NOT EXISTS idx_telemetry_msg_id
ON telemetry_readings (msg_id);

```

Kôd 4.8 Migracija sheme i stvaranje hipertablice.

Migracije izvršava alat Flyway [18] pri pokretanju svakog servisa. Budući da oba servisa dijele istu bazu, svaki vodi vlastitu povijest migracija u zasebnoj tablici (flyway_schema_history_ingestion odnosno flyway_schema_history_alerting), čime se izbjegavaju sukobi. Automatsko stvaranje sheme iz JPA entiteta isključeno je (ddl-auto: none) jer je shema u potpunosti pod nadzorom Flyway migracija.

Entitet `TelemetryReading` preslikan je na istoimenu tablicu. Hibernate od svakog entiteta zahtijeva identifikator, pa polje `msgId` nosi anotaciju `@Id` iako u bazi ne postoji odgovarajuće jedinstveno ograničenje. Ta je anotacija isključivo zahtjev razvojnog okvira i ne odražava se kao ograničenje u shemi. Entitet `AlertEvent` umjesto toga koristi surogatni ključ uz `@GeneratedValue`. Lombok anotacija `@Data` generira metode pristupa, dok `@Column(... TIMESTAMPTZ)` osigurava ispravno preslikavanje vremenskog tipa. Entitet `TelemetryReading` prikazan je isječkom (Kôd 4.9).

```

@Entity
@Table(name = "telemetry_readings")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class TelemetryReading {

    @Id
    private String msgId;

    private String deviceId;

    private String sensor;

    private double value;

    private String unit;

    private long timestamp;

    @Column(name = "received_at",
            columnDefinition = "TIMESTAMPPTZ")
    private Instant receivedAt;
}

```

Kôd 4.9 JPA entitet očitania telemetrije.

5. Infrastruktura i pokretanje

Cjelokupno poslužiteljsko okruženje sustava opisano je deklarativno i podiže se jednom naredbom (`docker compose up --build -d`), čime je razvojni postupak deterministički i lako ponovljiv. Ovo poglavlje opisuje konfiguraciju alata Docker Compose [19] i izgradnju slika mikroservisa, postavke posrednika Eclipse Mosquitto i pohrane Redis u radnoj memoriji te postupak pokretanja i provjere ispravnosti sustava.

5.1. Docker Compose konfiguracija

Datoteka `docker-compose.yml` okuplja šest servisa: posrednika `mosquitto`, pohranu `redis` u radnoj memoriji, bazu `postgres`, dva mikroservisa (`ingestion-service` i `alerting-service`) te pomoćni `webhook-mock`. Posrednik, baza, Redis i pomoćni servis pokreću se iz gotovih javnih slika (direktiva `image`), dok se dva mikroservisa grade lokalno iz vlastitog opisa (direktiva `build`). Suvremena specifikacija alata Docker Compose više ne zahtijeva polje `version`, pa ono nije navedeno.

Pravilan redoslijed pokretanja osiguran je kombinacijom mehanizama `depends_on` i `healthcheck`. Zdravstvene provjere definirane su za `redis` (naredba `redis-cli ping`) i `postgres` (naredba `pg_isready`), pa oba mikroservisa preko uvjeta `service_healthy` čekaju da baza i Redis budu spremni, a posrednika čekaju uvjetom `service_started`. Time se izbjegavaju pogreške veze pri prvom pokretanju. Politika ponovnog pokretanja postavljena je na `unless-stopped` za infrastrukturne servise i na `on-failure` za mikroservise.

Servisi se međusobno pronalaze po nazivima unutar zajedničke mreže koju Compose stvara automatski, pa se adrese ubrizgavaju varijablama okoline: mikroservisi se na posrednika spajaju preko `tcp://mosquitto:1883`, na bazu preko `SPRING_DATASOURCE_URL` usmjerenog na poslužitelj `postgres`, a servis za upozorenja `webhook` obavijesti šalje na adresu iz `ALERTING_WEBHOOK_URL`. Ulazi servisa izloženi su i prema lokalnom računalu radi lakšeg otkrivanja pogrešaka. Trajni se podaci čuvaju u imenovanim svescima (engl. *named volumes*) `postgres-data`, `mosquitto-data` i `mosquitto-log`, dok Redis namjerno nema svezak jer su njegovi ključevi prolazni i ne moraju preživjeti ponovno pokretanje.

Datoteka `docker-compose.yml` prikazana je isječkom (Kôd 5.1):

```

services:

  mosquitto:
    image: eclipse-mosquitto:2
    ports:
      - "1883:1883"
    volumes:
      -
./broker/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - mosquitto-data:/mosquitto/data
      - mosquitto-log:/mosquitto/log
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    restart: unless-stopped

  postgres:
    image: timescale/timescaledb:latest-pg15
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: iot_telemetry
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
        "pg_isready -U postgres -d iot_telemetry"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

```

```

ingestion-service:
  build: ./services/ingestion-service
  ports:
    - "8081:8081"
  environment:
    MQTT_BROKER: tcp://mosquitto:1883
    SPRING_DATASOURCE_URL:
jdbc:postgresql://postgres:5432/iot_telemetry
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: postgres
    SPRING_DATA_REDIS_HOST: redis
    SPRING_DATA_REDIS_PORT: "6379"
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
    mosquitto:
      condition: service_started
  restart: on-failure

alerting-service:
  build: ./services/alerting-service
  ports:
    - "8082:8082"
  environment:
    MQTT_BROKER: tcp://mosquitto:1883
    SPRING_DATASOURCE_URL:
jdbc:postgresql://postgres:5432/iot_telemetry
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: postgres
    SPRING_DATA_REDIS_HOST: redis
    SPRING_DATA_REDIS_PORT: "6379"
    ALERTING_WEBHOOK_URL: http://webhook-mock:9000/alert
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
    mosquitto:

```

```

        condition: service_started
        restart: on-failure

webhook-mock:
  image: mendhak/http-https-echo:latest
  ports:
    - "9000:9000"
  environment:
    HTTP_PORT: "9000"
  restart: unless-stopped

volumes:
  mosquitto-data:
  mosquitto-log:
  postgres-data:

```

Kôd 5.1 Konfiguracija okruženja u datoteci docker-compose.yml.

Dva su mikroservisa upakirana u slike pomoću višefaznog opisa (`Dockerfile`). Prva faza temelji se na slici `maven:3.9-eclipse-temurin-17` i prevodi projekt u izvršni JAR (`mvn package`), pri čemu se ovisnosti dohvaćaju u zaseban sloj naredbom `dependency:go-offline` radi učinkovitijeg predmemoriranja pri ponovnoj izgradnji. Druga faza polazi od znatno manje slike `eclipse-temurin:17-jre` (sadrži samo izvršno okruženje Jave, bez alata Maven i razvojnog kompleta) i preuzima isključivo izgrađeni JAR. Time je konačna slika manja i sadrži manju površinu za napad. Oba mikroservisa koriste istovjetan `Dockerfile`. Sam višefazni `Dockerfile` prikazan je isječkom (Kôd 5.2).

```

# Stage 1 – build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q
COPY src ./src
RUN mvn package -DskipTests -q

# Stage 2 – runtime
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Kôd 5.2 Višefazni Dockerfile mikroservisa.

5.2. Konfiguracija Mosquitto posrednika

Posrednik se pokreće iz službene slike Eclipse Mosquitto 2.x [20] uz prilagođenu datoteku `mosquitto.conf`. Osluškuje na ulazu `listener 1883` na svim sučeljima i dopušta anonimni pristup (`allow_anonymous true`), bez TLS-a i provjere autentičnosti, što je prihvatljivo jer sve komponente rade na lokalnoj mreži ili unutar Docker okruženja. Za javnu primjenu nužno bi bilo dodati TLS i datoteku s lozinkama, na što se vraća Zaključak.

Ključna postavka za pouzdanost jest perzistencija (`persistence true` uz `persistence_location /mosquitto/data/` i `autosave_interval 30`), kojom posrednik stanje sesija te poruke zadržane za nedostupne pretplatnike sprema na disk i tako preživljava ponovno pokretanje. Ta postavka izravno omogućuje scenarije T4 (ponovno pokretanje posrednika) i T5 (kvar servisa za unos) iz poglavlja 6. Uz to je veličina reda poruka za nedostupne pretplatnike podignuta na `max_queued_messages 100000` (zadano 1000), kako mikroservis koji je nedostupan tijekom cijelog testa kvara ne bi gubio poruke zbog prelijevanja reda.

Zapisivanje je usmjereno istovremeno u datoteku (`/mosquitto/log/mosquitto.log`) i na standardni izlaz (`stdout`), pa je dnevnik dostupan i kroz montirani svezak i izravno preko Docker zapisnika. Najveći broj istovremenih veza ograničen je na `max_connections 100`, što je dostatno za opisano okruženje.

Sadržaj datoteke `mosquitto.conf` prikazana je isječkom (Kôd 5.3):

```

listener 1883 0.0.0.0
allow_anonymous true

persistence true
persistence_location /mosquitto/data/
autosave_interval 30

max_queued_messages 100000

log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
log_type all

max_connections 100

```

Kôd 5.3 Konfiguracija posrednika Mosquitto.

5.3. Konfiguracija Redis pohrane

Za deduplikacijski sloj koristi se službena slika `redis:7-alpine` bez prilagođene konfiguracijske datoteke, što je za lokalni razvoj dostatno. Servis je izložen na uobičajenom ulazu `6379`, a njegovu spremnost potvrđuje zdravstvena provjera naredbom `redis-cli ping`. Za razliku od baze i posrednika, Redis namjerno nema trajni svezak jer su deduplikacijski ključevi prolazni (vrijeme isteka `3600` sekundi) i ne moraju preživjeti ponovno pokretanje.

U produkcijskom bi okruženju bilo uputno ograničiti memoriju postavkama `maxmemory 256mb` i `maxmemory-policy allkeys-lru`, čime bi se pri približavanju granici najstariji ključevi automatski uklanjali. Takvo ograničenje uskladilo bi se s procjenom memorijskog zauzeća iz poglavlja 1.5 i spriječilo nekontroliran rast u uvjetima vršnog opterećenja.

5.4. Pokretanje i verifikacija sustava

Cjelokupni se sustav podiže naredbom `docker compose up --build -d`, koja najprije izgrađuje slike dvaju mikroservisa, a zatim u pozadini pokreće svih šest servisa. Zahvaljujući zdravstvenim provjerama i ovisnostima, mikroservisi se pokreću tek kad su baza i Redis spremni. Trenutno stanje svih spremnika provjerava se naredbom `docker compose ps`.

Pokretanje sustava i osnovne provjere:


```
docker compose up --build -d
docker compose ps
mosquitto_sub -t "telemetry/#" -v
```

Ispravnost toka podataka provjerava se na nekoliko razina. Naredbom `mosquitto_sub` uz pretplatu na uzorak `telemetry/#` može se uživo pratiti promet koji uređaji objavljuju na posrednika. Trajna pohrana potvrđuje se jednostavnim brojačkim upitom nad tablicom `telemetry_readings`, dok se primljene obavijesti pojavljuju u dnevniku servisa `webhook-mock` na ulazu 9000, dostupnom naredbom `docker compose logs`.

```
SELECT COUNT(*) FROM telemetry_readings;
```

Nakon pokretanja okruženja uređaji `esp32-001` i `esp32-002` počinju objavljivati mjerenja čim dovrše sinkronizaciju vremena i kad je prekidač za objavu uključen (poglavlje 3). Time je sustav u stanju u kojem se mogu izvoditi sve eksperimentalne studije opisane u poglavlju 6.

6. Testiranje i rezultati

Ovo poglavlje predstavlja eksperimentalnu evaluaciju sustava. Najprije se opisuje metodologija mjerenja (generator opterećenja, mjerene veličine i način njihova izračuna), a zatim se kroz šest scenarija (T1 do T6) vrednuju performanse i pouzdanost: stabilno opterećenje, usporedba razina QoS, ispravnost deduplikacije, otpornost na kvar posrednika, otpornost na kvar servisa te deduplikacija pod visokim opterećenjem. Poglavlje završava objedinjenom analizom rezultata i osvrtom na uočena ograničenja.

6.1. Metodologija testiranja

Opterećenje sustava stvara generator opterećenja napisan u Pythonu (`load-generator.py`, biblioteka `paho-mqtt`). On simulira proizvoljan broj virtualnih uređaja, od kojih svaki radi u zasebnoj dretvi i objavljuje očitavanja temperature i vlažnosti na teme `telemetry/{device_id}/{sensor}`. Svaka poruka nosi jedinstveni `msg_id` u istom obliku kao na stvarnom uređaju ESP32, pa servisi za unos i upozorenja virtualne i fizičke uređaje obrađuju istovjetno. Generator isključivo objavljuje poruke i izvještava o broju poslanih, dok se latencija, gubitak i deduplikacija naknadno mjere nad bazom.

Ponašanje generatora određuju parametri `--devices` (broj uređaja), `--rate`, `--duration` (trajanje u sekundama) i `--qos` (razina kvalitete usluge). Bitno je istaknuti da je `--rate` broj ciklusa po sekundi po uređaju, a svaki ciklus proizvodi dvije poruke (temperatura i vlažnost). Ukupno ponuđeno opterećenje stoga iznosi $\text{broj_uređaja} \times \text{rate} \times 2$ poruka u sekundi, što treba imati na umu pri tumačenju konfiguracija pojedinih scenarija.

Mjerene veličine dijele se na veličine performansi i veličine pouzdanosti. Veličine performansi su: održivi protok (poruka u sekundi), ukupna latencija (medijan P50 te percentili P95 i P99, uz prosjek) i režija deduplikacijske provjere. Veličine pouzdanosti su: postotak gubitka poruka, ispravnost suzbijanja duplikata, potpunost i vrijeme nadoknade nakon kvara, dubina reda posrednika, ponašanje međuspremnik na uređaju te memorijski otisak Redis skupa viđenih poruka. Postotak gubitka definiran je kao $(\text{poslano} - \text{jedinstveno_pohranjeno}) / \text{poslano} \times 100$, a latencija kao razlika poslužiteljskog vremena primitka (`received_at`) i vremena mjerenja iz poruke (`timestamp`).

Za generator opterećenja vrijeme objave i `received_at` dijele isti domaćin (engl. *host*), pa nema pomaka sata i ta je latencija primarna mjera. Latencija stvarnog uređaja ESP32 dodatno nosi pogrešku NTP sinkronizacije. Uloge su podijeljene: generator opterećenja pokreće scenarije propusnosti i deduplikacije (T1, T2, T6), dok uređaj ESP32 demonstrira pouzdanost na stvarnom uređaju (T4). Pritom pouzdanost na uređaju osigurava međuspremnik s ponovnim spajanjem (ne MQTT-ov mehanizam neisporučenih poruka), a sloj posrednik-servis osigurava razina QoS 1 uz `cleanSession=false` i datotečnu perzistenciju (T5).

Radi usporedivosti svaki se scenarij pokreće iz čistog stanja: skripta `reset.ps1` prije svakog pokretanja prazni obje tablice (`TRUNCATE`) i Redis (`FLUSHDB`). Mjerne veličine računaju se SQL upitima okupljenima u datoteci `measure.sql`, koja nad bazom izračunava broj pohranjenih i jedinstvenih poruka, eventualne duplikate, percentile latencije, protok i raspon nadoknade. Latencijski percentili dobivaju se upitom prikazanim isječkom (Kôd 6.1).

```
SELECT
    ROUND(AVG(lat)::numeric, 1) AS avg_ms,
    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP
        (ORDER BY lat)::numeric, 1) AS p50_ms,
    ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP
        (ORDER BY lat)::numeric, 1) AS p95_ms,
    ROUND(PERCENTILE_CONT(0.99) WITHIN GROUP
        (ORDER BY lat)::numeric, 1) AS p99_ms,
    COUNT(*) AS samples
FROM (
    SELECT EXTRACT(EPOCH FROM received_at) * 1000
        - "timestamp" AS lat
    FROM telemetry_readings
    WHERE received_at > NOW() - INTERVAL '5 minutes'
) t;
```

Kôd 6.1 SQL upit za izračun percentila latencije.

Ispravnost deduplikacije provjerava se upitom koji grupira retke po `msg_id` i zadržava samo one koji se pojavljuju više puta. Ako taj upit ne vrati nijedan redak, u bazi nema duplikata. Scenarij stabilnog opterećenja (T1) ponovljen je tri puta i u tablici se uz pojedinačna pokretanja navodi prosjek, dok su scenariji pouzdanosti i deduplikacije (T2 do T6) izvedeni kao zasebna ciljana mjerenja iz čistog stanja.

6.2. T1: Stabilno opterećenje

Prvi scenarij utvrđuje osnovnu razinu performansi pri umjerenom, stabilnom opterećenju. Pet virtualnih uređaja objavljuje s dva ciklusa u sekundi (četiri poruke u sekundi po uređaju, ukupno oko 20 poruka u sekundi) tijekom 120 sekundi uz razinu QoS 1, što po pokretanju daje 2400 objavljenih poruka. Mjerenje je ponovljeno tri puta, a rezultati su prikazani u tablici (Tablica 6.1).

Tablica 6.1 Rezultati scenarija T1 (stabilno opterećenje).

Pokretanje	Objavljeno	Jedinstveno pohranjeno	Gubitak (%)	P50 (ms)	P95 (ms)	P99 (ms)	Protok (msg/s)
1	2400	2400	0,0	18,1	32,5	42,4	20,1
2	2400	2400	0,0	15,4	27,5	33,0	20,1
3	2400	2400	0,0	16,4	27,0	31,0	20,1
Prosjek	2400	2400	0,0	16,6	29,0	35,5	20,1

U sva tri pokretanja sve su objavljene poruke i pohranjene, uz nulti postotak gubitka. Prosječni percentil P99 od 35,5 ms (medijan P50 od 16,6 ms) znatno je unutar zahtjeva za obradom u stvarnom vremenu, a mali raspon između pokretanja (P99 od 31 do 42 ms) pokazuje da deduplikacijska provjera unosi zanemarivu i stabilnu režiju. Održivi protok pratio je ponuđenu brzinu (oko 20 poruka u sekundi) bez nakupljanja zaostatka, uz ravnomjernu raspodjelu po uređajima i senzorima.

6.3. T2: Usporedba razina QoS

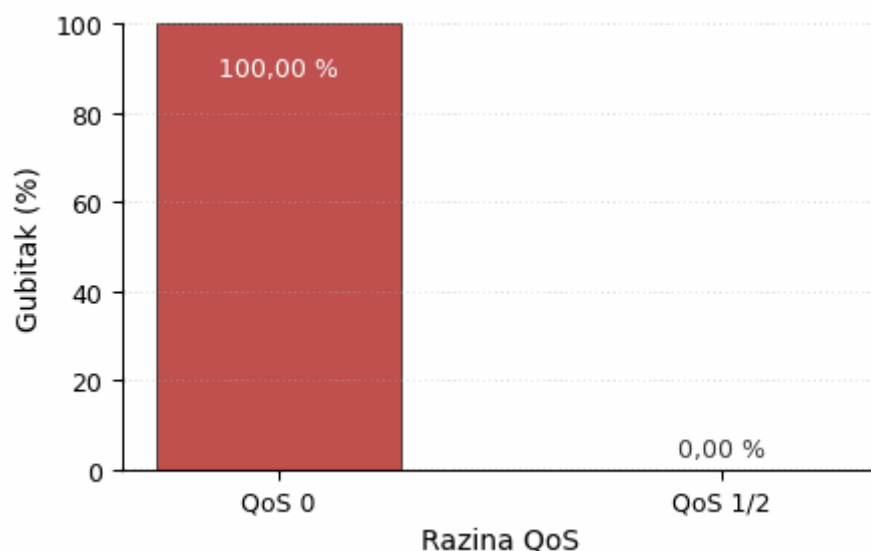
Drugi scenarij vrednuje granicu pouzdanosti isporuke između razina kvalitete usluge. Na zdravoj lokalnoj vezi MQTT promet putuje preko TCP-a, koji retransmitira na transportnom sloju, pa dok je potrošač spojen sve razine QoS bilježe gubitak blizu nule. Razlika između njih postaje vidljiva tek kad isporuke nema kome predati. Zato se za svaku razinu QoS sustav najprije dovede u čisto stanje, servis za unos koji je već pretplaćen (perzistentna sesija, `cleanSession=false`) zaustavi tako da je potrošač uredno odsutan, zatim se objavljuje oko deset sekundi s pet virtualnih uređaja (1000 poruka po pokretanju), nakon čega se potrošač ponovno pokreće i prazni zaostatak. Razine QoS 1 i QoS 2 su po gubitku istovjetne, stoga

se prikazuje usporedba QoS 0 razine naspram viših razina QoS 1/2. Rezultati su prikazani u tablici (Tablica 6.2).

Tablica 6.2 Usporedba razina QoS uz nedostupnog potrošača (scenarij T2).

QoS	Objavljeno	Jedinstveno pohranjeno	Gubitak (%)	Duplikata	Napomena
0	1000	0	100,00	0	posrednik ne čuva QoS 0 za nedostupnog pretplatnika
1/2	1000	1000	0,00	0	zadržano za perzistentnu sesiju i reproducirano pri ponovnom spajanju

Uz nedostupnog potrošača razina QoS 0 izgubila je sve poruke (100 %) jer ih posrednik isporučuje samo trenutno spojenim pretplatnicima i ne stavlja ih u red, dok je razina QoS 1/2 ostvarila nulti gubitak: posrednik je zaostatak zadržao za perzistentnu sesiju i reproducirao ga pri ponovnom spajanju, bez ijednog duplikata zahvaljujući Redis deduplikaciji. Time je opravdan odabir kombinacije QoS 1 i Redis deduplikacije: ona preživljava ispad potrošača koji QoS 0 ne preživljava i prijenos po načelu barem jednom pretvara u pohranu točno jednom, uz cijenu kašnjenja pri naknadnoj reprodukciji zaostatka. Postotak gubitka za QoS 0 nasuprot QoS 1/2 prikazan je na slici (Sl. 6.1).



Sl. 6.1 Postotak gubitka poruka uz nedostupnog potrošača: QoS 0 nasuprot QoS 1/2.

6.4. T3: Obrada duplikata

Treći scenarij izravno provjerava ispravnost deduplikacije. Ista poruka s fiksnim identifikatorom `msg_id = dup-test-001` objavljena je pet puta zaredom uz razinu QoS 1, čime se oponaša ponovljena isporuka. Očekuje se da bude pohranjen točno jedan redak, a preostale četiri isporuke odbačene. Rezultati su prikazani u tablici (Tablica 6.3).

Tablica 6.3 Rezultati scenarija T3 (obrada duplikata).

Mjera	Očekivano	Izmjereno
Pohranjenih redaka za <code>dup-test-001</code>	1	1
Zapisa o odbačenom duplikatu	4	4

Prva je poruka pohranjena, a četiri ponovljene isporuke prepoznao je Redis skup viđenih poruka i odbacio ih, čime je potvrđena idempotentna obrada. Nijedan duplikat nije dospio u hipertablicu, što potvrđuje da jamstvo deduplikacije na aplikacijskom sloju vrijedi neovisno o razini QoS posrednika.

6.5. T4: Kvar posrednika

Četvrti scenarij vrednuje pouzdanost na strani uređaja, koju osiguravaju kružni međuspremnik u radnoj memoriji i neblokirajuće ponovno spajanje (poglavlje 3), a ne

MQTT-ov mehanizam neisporučenih poruka. Tijekom objavljivanja sa stvarnog uređaja ESP32 (prekidač uključen, oko 0,4 poruke u sekundi) posrednik se nasilno zaustavlja usred toka i ponovno pokreće nakon isteka prozora kvara. Pri kapacitetu `OUTBOX_CAPACITY` od 256 unosa izvedene su dvije varijante: kraći prekid ispod kapaciteta i dulji prekid iznad kapaciteta. Rezultati su prikazani u tablici (Tablica 6.4).

Tablica 6.4 Rezultati scenarija T4 (kvar posrednika, međuspremnik uređaja).

Varijanta	Prekid (s)	Vršna dubina međuspremnika	[Outbox] FULL	Očitavanja u prozoru	Pohranjeno	Gubitak (%)
Ispod kapaciteta	300	118	ne	118	118	0,0
Iznad kapaciteta	900	256	da	356	256	28,1

Dok je posrednik bio nedostupan, uređaj je očitavanja nakupljao u međuspremniku i reproducirao ih po ponovnom spajanju, uz nulti gubitak za prekid od 300 sekundi koji je ostao ispod kapaciteta `OUTBOX_CAPACITY`. Pri prekidu od 900 sekundi kružni se spremnik popunio i, kako je i predviđeno, odbacivao najstarije unose, dajući ograničen i predvidljiv gubitak (28,1 %) umjesto rušenja ili nekontroliranog rasta memorije. Zapisi `[Outbox] FULL` točno označavaju trenutak u kojem je odbacivanje najstarijih unosa započelo.

6.6. T5: Kvar servisa za unos

Peti scenarij vrednuje pouzdanost na sloju posrednika. Servis za unos zaustavlja se dok generator objavljuje poruke (dva uređaja s tri ciklusa u sekundi tijekom 30 sekundi, ukupno 360 poruka), nakon čega se servis ponovno pokreće i nadoknađuje zaostatak. Pouzdanost ovdje počiva na tome da posrednik za nedostupnog trajnog pretplatnika čuva poruke (`cleanSession=false` uz perzistenciju Mosquitta). Rezultati su prikazani u tablici (Tablica 6.5).

Tablica 6.5 Rezultati scenarija T5 (kvar servisa za unos).

Mjera	Vrijednost
Objavljeno za vrijeme nedostupnosti	360
Posrednik zadržao (približno, \$SYS)	416

Mjera	Vrijednost
Pohranjeno nakon ponovnog pokretanja	360
Potpunost nadoknade (pohranjeno == objavljeno)	da (360 / 360)
Duplikata nakon nadoknade	0
Vrijeme nadoknade (s)	3,2

Svaka poruka objavljena tijekom nedostupnosti isporučena je i pohranjena nakon ponovnog pokretanja, bez gubitka i bez duplikata, čime je potvrđen sloj pouzdanosti utemeljen na čuvanju poruka u posredniku za trajnog pretplatnika. Nadoknada je dovršena za 3,2 sekunde nakon što se servis ponovno spojio i obnovio pretplatu putem povratnog poziva `connectComplete`. Brojač `$SYS` pokazuje veću vrijednost (416) od broja poruka iz testa jer odražava sve poruke koje posrednik u tom trenutku drži u spremištu (zadržane poruke, `$SYS` teme i druge sesije), pa se ne preslikava jedan na jedan na promet ovog scenarija. End-to-end latencija nadoknađenih poruka pritom je visoka (prosječno oko 53 sekunde) jer odražava vrijeme koje su poruke provele u redu posrednika tijekom nedostupnosti servisa, a ne troškove same obrade.

6.7. T6: Deduplikacija pod opterećenjem

Šesti scenarij provjerava deduplikaciju pri visokom opterećenju, gdje je učestalost ponovljenih isporuka QoS 1 veća. Dvadeset virtualnih uređaja objavljuje s deset ciklusa u sekundi tijekom 60 sekundi (ukupno ponuđeno opterećenje oko 385 poruka u sekundi, 23566 poruka u ovom pokretanju). Mjerenje se provodi tek nakon što se zaostatak u potpunosti isprazni, a uz mjere performansi bilježi se i memorijski otisak Redis skupa viđenih poruka. Rezultati su prikazani u tablici (Tablica 6.6).

Tablica 6.6 Rezultati scenarija T6 (deduplikacija pod opterećenjem).

Mjera	Vrijednost
Objavljeno	23566
Jedinstveno pohranjeno	23566
Gubitak (%)	0,0

Mjera	Vrijednost
Duplikata u bazi	0
Održivi protok (msg/s)	191,0
Vrijeme nadoknade (s)	123,4
P50 / P99 latencija (ms)	48725 / 63031
Redis DBSIZE (oba servisa)	47132
Redis zauzeće memorije	6,67 MB

Pri ukupnom ponuđenom opterećenju od oko 385 poruka u sekundi deduplikacijski je sloj suzbio sve ponovljene isporuke (nula duplikata u bazi) i pohranio svih 23566 poruka točno jednom (nulti gubitak). Budući da ponuđeno opterećenje nadmašuje propusnost jednonitnog potrošača (oko 191 poruka u sekundi), posrednik je višak zadržao u redu perzistentne QoS 1 sesije, a servis ga je ispraznio tijekom 123,4 sekunde. Taj se protupritisak (engl. *backpressure*) očituje kao visoka ukupna latencija (P50 oko 48,7 s, P99 oko 63,0 s) mjerena od trenutka objave, jer je većina poruka čekala u redu posrednika prije pohrane, a ne kao trošak same obrade. Redis skup viđenih poruka držao je 47132 ključa (svaki jedinstveni `msg_id` bilježi se jednom po servisu, dakle dvostruko) uz zauzeće od 6,67 MB, što je u skladu s analitičkom procjenom memorijskog otiska iz poglavlja 1.5. Rezultat pokazuje postupnu degradaciju pri približno dvostrukom preopterećenju: sustav trampi latenciju za trajnost, bez gubitka i bez duplikata umjesto odbacivanja podataka.

6.8. Analiza rezultata i zaključci testiranja

Objedinjeni rezultati potvrđuju glavnu tezu rada: kombinacija razine QoS 1, aplikacijske deduplikacije i perzistentne sesije posrednika (`cleanSession=false`) daje nulti gubitak poruka u svim ispitanim scenarijima, uz pouzdanost na strani uređaja koja vrijedi do kapaciteta međuspremnik. Ispravnost deduplikacije potvrđena je i izravnim testom duplikata (T3) i pod visokim opterećenjem (T6), u oba slučaja bez ijednog duplikata u bazi.

S gledišta performansi, u scenarijima u kojima potrošač prati ponuđeno opterećenje (T1 i T2) medijan latencije ostao je ispod 20 ms, a percentil P99 ispod 100 ms, iz čega slijedi da Redis i PostgreSQL pri tim opterećenjima nisu usko grlo. U scenarijima u kojima je servis bio nedostupan (T5) ili je ponuđeno opterećenje nadmašilo propusnost jednonitnog potrošača (T6) ukupna latencija raste na red veličine sekundi, no ta vrijednost odražava

vrijeme čekanja poruka u redu posrednika, a ne troškove obrade. Sustav pritom degradira postupno, zadržavajući nulti gubitak i nulu duplikata umjesto odbacivanja podataka. Izmjereni memorijski otisak Redis skupa viđenih poruka potvrdio je analitičku procjenu iz poglavlja 1.5, čime je analitički model deduplikacijskog sloja eksperimentalno opravdan.

Testiranjem su potvrđena i dva ranije opisana rubna slučaja, koja se navode kao ograničenja zanemariva u praksi. Prvo, ponovno pokretanje uređaja vraća brojač na nulu, pa bi unutar Redis TTL prozora teoretski mogao nastati lažan duplikat, no istodobna promjena vremenske oznake čini to izrazito malo vjerojatnim. Drugo, između upisa u bazu i oznake u Redis-u (`markProcessed` nakon `save`) postoji uzak vremenski prozor u kojem bi pad servisa mogao dopustiti dodatni upis, jer baza zbog ograničenja hipertablica nema jedinstveno ograničenje nad `msg_id`. Ručno potvrđivanje primitka uvedeno u poglavlju 4 čini ponovnu isporuku takve poruke izvjesnom (posrednik je isporučuje jer izostaje potvrda), pa uklanja gubitak pri prolaznim pogreškama, ali sam prozor dvostrukog upisa ne uklanja. Oba su slučaja u izvedenim mjerenjima ostala bez učinka i predstavljaju smjerove za buduće ojačavanje sustava.

Zaključak

U radu je implementiran prototip sustava za pouzdanu obradu IoT telemetrije u mikroservisnoj arhitekturi vođenoj događajima, koji povezuje uređaje ESP32, posrednika Eclipse Mosquitto, dva mikroservisa razvijena u okviru Spring Boot, Redis te bazu PostgreSQL s ekstenzijom TimescaleDB. Ključni je doprinos slojevita strategija pouzdanosti: razina QoS 1 jamči dostavu barem jednom, dok moguće duplikate uklanja aplikacijski sloj pomoću Redis skupa viđenih poruka i jedinstvenog identifikatora `msg_id`, čime je idempotentnost ostvarena na aplikacijskom sloju. Pouzdanost dodatno učvršćuju perzistentna sesija posrednika, trajna pohrana stanja klijenta, ručno potvrđivanje primitka tek nakon uspješne obrade te međuspremnik u radnoj memoriji uređaja ESP32.

Eksperimentalno vrednovanje kroz šest scenarija potvrdilo je djelotvornost pristupa: kombinacija razine QoS 1, aplikacijske deduplikacije i perzistentne sesije ostvarila je nulti gubitak poruka i ispravno suzbijanje duplikata u svim uvjetima, uključujući kvar posrednika, kvar servisa i visoko opterećenje. Pri stabilnom opterećenju medijan ukupne latencije ostao je ispod 20 ms, a P99 ispod 100 ms, dok je pod preopterećenjem ili nedostupnošću servisa sustav postupno degradirao zadržavajući nulti gubitak. Redis i PostgreSQL pritom nisu bili usko grlo.

Prepoznata su i određena ograničenja, zanemariva u praksi: budući da se brojač poruka drži u radnoj memoriji uređaja, ponovno pokretanje unutar prozora isteka Redis ključeva teoretski bi moglo proizvesti lažan duplikat, a uzak vremenski prozor između upisa u bazu i bilježenja oznake u Redis mogao bi pri istodobnom padu servisa dopustiti dodatni upis. Oba su slučaja u mjerenjima ostala bez učinka. Smjerovi nadogradnje uključuju TLS i provjeru autentičnosti na posredniku, zamjenu izravnih MQTT pretplata sabirnicom poput Apache Kafke ili RabbitMQ-a radi horizontalnog skaliranja, prelazak s Docker Composea na Kubernetes te napredne mogućnosti TimescaleDB-a poput kontinuiranih agregata i politika sažimanja podataka.

Zaključno, rad pokazuje da se promišljenim kombiniranjem provjerenih tehnologija i jasno razgraničenih slojeva pouzdanosti može izgraditi sustav koji telemetrijske podatke od senzora do trajne pohrane dostavlja pouzdano, idempotentno i s predvidljivom latencijom. Premda je riječ o prototipu, predložene nadogradnje ocrtavaju jasan put prema sustavu primjerenom i za zahtjevnija produkcijska okruženja.

Literatura

- [1] Rose, K., Eldridge, S., Chapin, L., *The Internet of Things: An Overview*, The Internet Society (ISOC), 2015. Poveznica: <https://www.internetsociety.org/wp-content/uploads/2017/08/ISOC-IoT-Overview-20151221-en.pdf>; pristupljeno 19. svibnja 2026.
- [2] OASIS, *MQTT Version 3.1.1 OASIS Standard*, 2014. Poveznica: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>; pristupljeno 6. svibnja 2026.
- [3] OASIS, *MQTT Version 5.0 OASIS Standard*, 2019. Poveznica: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>; pristupljeno 23. svibnja 2026.
- [4] Fowler, M., Lewis, J., *Microservices*, martinowler.com, 2014. Poveznica: <https://martinfowler.com/articles/microservices.html>; pristupljeno 11. svibnja 2026.
- [5] Richardson, C., *Microservices Pattern: Microservice Architecture*, microservices.io. Poveznica: <https://microservices.io/>; pristupljeno 28. svibnja 2026.
- [6] Redis Ltd., *Redis Documentation*. Poveznica: <https://redis.io/docs/latest/>; pristupljeno 4. lipnja 2026.
- [7] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Poveznica: <https://www.postgresql.org/docs/>; pristupljeno 14. svibnja 2026.
- [8] VMware (Broadcom), *Spring Data JPA i Spring Data Redis, Reference Documentation*. Poveznica: <https://spring.io/projects/spring-data>; pristupljeno 9. lipnja 2026.
- [9] Timescale, *TimescaleDB Documentation, Hypertables*. Poveznica: <https://docs.timescale.com/>; pristupljeno 2. svibnja 2026.
- [10] Eclipse Foundation, *Eclipse Paho Java Client, dokumentacija*. Poveznica: <https://eclipse.dev/paho/>; pristupljeno 27. svibnja 2026.
- [11] Arduino, *Arduino Documentation*. Poveznica: <https://docs.arduino.cc/>; pristupljeno 12. lipnja 2026.
- [12] Espressif Systems, *ESP32 Series Datasheet*. Poveznica: <https://www.espressif.com/en/products/socs/esp32>; pristupljeno 16. svibnja 2026.
- [13] PlatformIO, *PlatformIO Documentation*. Poveznica: <https://docs.platformio.org/>; pristupljeno 8. svibnja 2026.
- [14] O'Leary, N., *PubSubClient, Arduino Client for MQTT*. Poveznica: <https://github.com/knolleary/pubsubclient>; pristupljeno 30. svibnja 2026.
- [15] Blanchon, B., *ArduinoJson*. Poveznica: <https://arduinojson.org/>; pristupljeno 3. lipnja 2026.

- [16] Adafruit Industries, *DHT sensor library (DHT11)*. Poveznica: <https://github.com/adafruit/DHT-sensor-library>; pristupljeno 21. svibnja 2026.
- [17] VMware (Broadcom), *Spring Boot Reference Documentation*. Poveznica: <https://docs.spring.io/spring-boot/>; pristupljeno 7. lipnja 2026.
- [18] Redgate, *Flyway Documentation*. Poveznica: <https://documentation.red-gate.com/flyway>; pristupljeno 25. svibnja 2026.
- [19] Docker Inc., *Docker Compose Documentation*. Poveznica: <https://docs.docker.com/compose/>; pristupljeno 13. lipnja 2026.
- [20] Eclipse Foundation, *Eclipse Mosquitto, An open source MQTT broker, dokumentacija*. Poveznica: <https://mosquitto.org/documentation/>; pristupljeno 10. svibnja 2026.

Izjava o korištenju umjetne inteligencije

Pri izradi ovog rada korištena je umjetna inteligencija. Korišteni su sljedeći alati: Claude Code (model Opus 4.8, tvrtka Anthropic) za organizaciju i refaktoriranje izvornog koda, pripremu pomoćnih skripti te oblikovanje i jezično ujednačavanje teksta dokumenta, kao i Perplexity AI (tvrtka Perplexity), uz modele Sonnet 4.6 (tvrtka Anthropic) i ChatGPT 5.4 (tvrtka OpenAI), za pretraživanje literature, prikupljanje i provjeru izvora te pojašnjavanje tehničkih pojmova pri istraživanju, kao i za jezično oblikovanje teksta. Alati su korišteni tijekom prve polovice 2026. godine.

Osvrt na korištenje umjetne inteligencije: Umjetna inteligencija korištena je kao pomoćni alat za oblikovanje, jezičnu doradu i strukturiranje teksta, dok su svi tehnički sadržaji, arhitektonske odluke, implementacija sustava te provedba i tumačenje mjerenja rezultat vlastitog rada. Sav generirani sadržaj pregledan je i prilagođen, a tehnička točnost provjerena je usporedbom s izvornim kodom i izmjerenim rezultatima. Nisu uočene značajne halucinacije koje bi ušle u konačni tekst.

Izjava o korištenju umjetne inteligencije: Potvrđujem da sam tijekom izrade ovog diplomskog rada koristio umjetnu inteligenciju u skladu s Politikom primjerenog korištenja umjetne inteligencije na Fakultetu elektrotehnike i računarstva, te da umjetna inteligencija nije zamijenila moje kritičko razmišljanje niti moj doprinos.

Skraćenice

DHT	<i>Digital Humidity and Temperature</i>	digitalni senzor vlažnosti i temperature
DTO	<i>Data Transfer Object</i>	objekt za prijenos podataka
EDA	<i>Event-Driven Architecture</i>	arhitektura vođena događajima
GPIO	<i>General-Purpose Input/Output</i>	ulazno-izlazna nožica opće namjene
HTTP	<i>HyperText Transfer Protocol</i>	protokol za prijenos hiperteksta
IoT	<i>Internet of Things</i>	Internet stvari
JAR	<i>Java Archive</i>	Java arhiva
JPA	<i>Java Persistence API</i>	Java sučelje za perzistenciju
JRE	<i>Java Runtime Environment</i>	izvršno okruženje Jave
JSON	<i>JavaScript Object Notation</i>	JavaScript zapis objekata
LRU	<i>Least Recently Used</i>	najdulje nekoristeni
MQTT	<i>Message Queuing Telemetry Transport</i>	protokol za prijenos telemetrije
NTP	<i>Network Time Protocol</i>	mrežni protokol za sinkronizaciju vremena
QoS	<i>Quality of Service</i>	kvaliteta usluge
SQL	<i>Structured Query Language</i>	strukturirani upitni jezik
TCP	<i>Transmission Control Protocol</i>	protokol za upravljanje prijenosom
TLS	<i>Transport Layer Security</i>	sigurnost transportnog sloja
TOCTOU		<i>Time-Of-Check To Time-Of-Use</i> vrijeme provjere naspram vremena uporabe
TTL	<i>Time-To-Live</i>	vrijeme do isteka
UUID	<i>Universally Unique Identifier</i>	univerzalno jedinstveni identifikator